



Joint optimization of storage assignment and order batching in robotic mobile fulfillment system with dynamic storage depth and surplus items

Zhi Liu^{a,b}, Jiansha Lu^{a,*}, Chenhao Ren^c, Jun Chen^a, Zhilong Xu^a, Guoli Zhao^{a,d}

^a College of Mechanical Engineering, Zhejiang University of Technology, Hangzhou 310023, China

^b School of Physics and Electronic Engineering, Yancheng Teachers University, Yancheng 224007, China

^c School of Economics and Management, Zhejiang University of Science and Technology, Hangzhou 310023, China

^d Bosch Power Tools (China) Co., Ltd., Hangzhou 310052, China

ARTICLE INFO

Keywords:

Storage location assignment
Order batching
Robotic mobile fulfillment system
Dynamic system

ABSTRACT

This paper studies the joint optimization of storage location assignment and order batching in robotic mobile fulfillment systems (RMFS), considering dynamic storage depth and surplus items. Firstly, a joint optimization model of storage location assignment and order batching is established. The model is divided into two stages. The first stage is the item location assignment optimization model, which is used to describe the types and quantities of items placed in each slot on each pod. The second stage is the joint optimization model of pod location assignment and order batching, which is used to describe the coordinates of each pod, the number of order batches, and the order combinations contained in each batch. Considering the vast solution space and the numerous constraints of the constructed model, a two-stage greedy variable neighborhood simulated annealing algorithm (TGVNSA) is introduced to address these challenges. Finally, numerical experiments prove that the algorithm can effectively solve the established model. TGVNSA is evaluated against two conventional methods: variable neighborhood search and adaptive genetic algorithms, focusing on metrics such as pod retrieval times, comprehensive picking costs, CPU time, and CQ value (comprehensive quality in the item location assignment). The findings demonstrate that TGVNSA boasts superior comprehensive performance. Compared with other commonly used strategies in the field such as random, classification, and optimal relevance, the method proposed in this paper demonstrates superior optimization performance, particularly when considering dynamic storage depth and surplus items. Moreover, this paper also proves that, under the same combination of strategies, the joint optimization method proposed in this paper reduces the comprehensive picking costs by 11.46% compared to the separate optimization approach.

1. Introduction

The rapid development of the mobile robot industry has brought significant changes in the field of warehousing and logistics. The storage and retrieval systems have evolved from the traditional picker-to-parts model to the parts-to-picker model. In 2012, Amazon acquired Kiva and applied its robots to its distribution centers. Since then, robotic mobile fulfillment systems (RMFS) have been officially applied in large-scale systems (Boysen et al. 2019). RMFS has greatly improved the efficiency of the labor-intensive sorting work, reduced the error rate, reduced costs, improved safety, and increased space utilization. At the same time, RMFS also has high flexibility and scalability. Mobile robots equipped with sensors and cameras can collect a large amount of data,

which helps optimize warehouse layout, inventory management, and operational processes, helping enterprises adapt to constantly changing market demands (Barros and Nascimento 2021). Given its unique advantages, RMFS is gradually applied in various types of distribution centers, e-commerce warehouses, material supermarkets, and other systems that integrate warehousing and sorting all over the world.

The operation of RMFS mainly includes the following several processes: First, the items are put into storage. The items are placed on the pods (shelves) according to the established strategy. The robots carry the pods to the designated storage area and place them according to a certain storage strategy. After the customer places an order, the order processing operation is carried out. The warehouse management system assigns the picking tasks in the order to the robots. The robots move to

* Corresponding author.

E-mail address: ljs@zjut.edu.cn (J. Lu).

<https://doi.org/10.1016/j.cie.2024.110767>

Received 4 July 2024; Received in revised form 28 October 2024; Accepted 26 November 2024

Available online 4 December 2024

0360-8352/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

the corresponding pod locations according to the task sequence, and bring the pods to the picking area, then the picking operation is carried out by manual workers or picking robots. Finally, packaging and verification, outbound, and distribution are conducted (D'Andrea et al. 2008). During the entire process, the system will continuously collect data, including picking time, robot operating status, order accuracy, etc., for future analysis and system optimization. RMFS has unparalleled advantages in picking efficiency compared to other systems, making it especially suitable for e-commerce warehouses or manufacturing material supermarkets (Yang et al. 2021).

Research related to RMFS can be divided into system analysis, design optimization, and operations planning and control. The operation planning can also be subdivided into four aspects: storage assignment, order batching, task allocation, and path planning (Azadeh et al. 2019). The schematic diagram of the RMFS operation process is shown in

Fig. 1. Conventional configurations of RMFS have been the subject of considerable research. In recent years, with rising land use costs, intensive storage has become a topic of significant practical importance. While there has been some research on dense RMFS (Yang et al. 2022), studies specifically addressing the storage location assignment problem remain relatively scarce.

Storage assignment is based on the characteristics and storage requirements, exploring the correlations between items, placing them in the corresponding pods in the optimal combination, and assigning the pods to reasonable positions. The purpose is to improve the utilization rate of warehouse space, optimize the picking efficiency, enhance the order processing capacity, and quickly adapt to changes in demand, etc. Storage assignment is crucial for ensuring the efficiency and safety of RMFS and adapting to the constantly changing business requirements (Roy et al. 2019). In actual operation, there are tens of thousands of types of items. Each type of item may be selected simultaneously with many other items, and the quantitative relationships are intricate and complex. Faced with the large-scale item combinations and quantity relationships reflected by massive orders, storage assignment aims to determine the types and quantities of items stored on each pod to meet the diverse order-picking demands as much as possible, thereby minimizing the number of pod movements required for order picking, so storage assignment is extremely difficult.

Order batching involves combining multiple orders for picking at the same time, which reduces repetitive tasks for robots or manual labor. The goal is to reduce the number of times pods need to be moved during order picking, as well as to minimize the walking distance or time for robots. The strategies used in order batching typically consider principles such as the attributes of the items, their priority, and their similarity (Boysen et al. 2017).

There has been considerable research on the optimization of individual stages in RMFS. However, joint optimization of two or more stages often leads to better results (Scholz et al. 2017; K. Zhang and Gao 2023). Storage assignment and order batching are both preliminary

preparations for task allocation. Therefore, the joint optimization of these two stages can provide better initial values for subsequent work.

The flexibility and scalability of the RMFS enable it to meet complex and varying demands and environments. However, there is relatively little research on RMFS that takes random factors and dynamic situations into account, yet it is particularly crucial at this stage. Therefore, this paper focuses on real-world scenarios and undertakes joint optimization of storage assignment and order batching for the RMFS, considering the dynamic changes of certain parameters. The dynamic parameters considered in this paper include storage depth, the surplus items in each pod, and the number of orders contained in each batch.

The main content of this paper is to address the problem of determining which items should be placed on which pods and where the pods should be positioned in the storage area. This problem is divided into two stages. The first stage involves the item location assignment, placing various items in different slots (storage positions in each pod), to minimize the number of pod retrieval times accessed during order fulfillment. The second stage focuses on the pod location assignment, incorporating order batching for joint optimization, aiming to minimize the comprehensive picking costs during the order fulfillment process. The schematic diagram of the two stages is shown in Fig. 2.

The storage depth is allowed to change dynamically in this paper, with different areas permitted to utilize varying depths. Meanwhile, during the order fulfillment process, the surplus items on each pod are recorded in real-time, allowing for a more accurate description of the number of retrieval times for each pod. Subsequently, an integer programming model is established to describe this problem, and a fast and effective heuristic algorithm is developed to solve it based on the characteristics of the problem.

The dynamic storage depth means that the storage depth can be flexibly selected according to actual needs. If the quantity of items to be stored is moderate and there is a high demand for efficient inbound and outbound operations, a single-depth storage configuration can be used. This configuration ensures that an adequate number of robots can operate simultaneously without causing conflicts or deadlocks, as illustrated in Fig. 3(a). If there are too many items to be stored, and some of them have a low turnover rate, then, certain areas can be set up as double-depth or multi-depth storage to increase the maximum capacity of racks, allowing it to have enough space to store items that meet the order requirements, as shown in Fig. 3(b).

The dynamic surplus items in each pod mean that the consumption of items during the picking process will be taken into account. The number of items that a pod can store is not infinite. As the picking operation proceeds, the storage capacity of each pod will decrease to varying degrees. Obviously, whether to consider the dynamic surplus items has a great impact on the optimization of all stages of RMFS. So, in the process of optimizing storage assignment and order batching, it is also necessary to consider dynamic surplus items in each pod.

Order batching can significantly reduce the number of pod retrievals

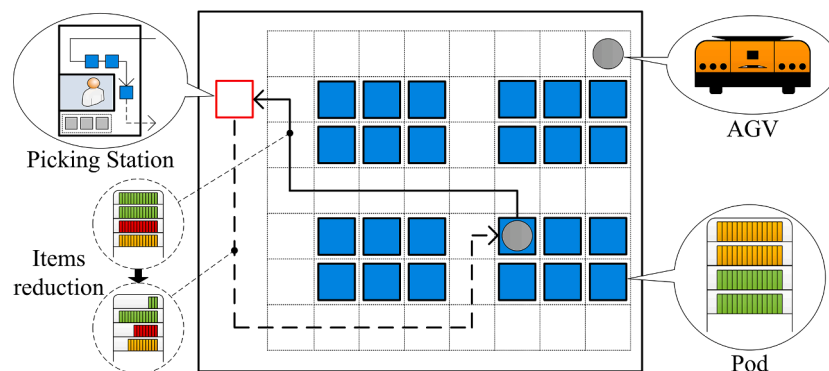


Fig. 1. Schematic diagram of RMFS operation process.

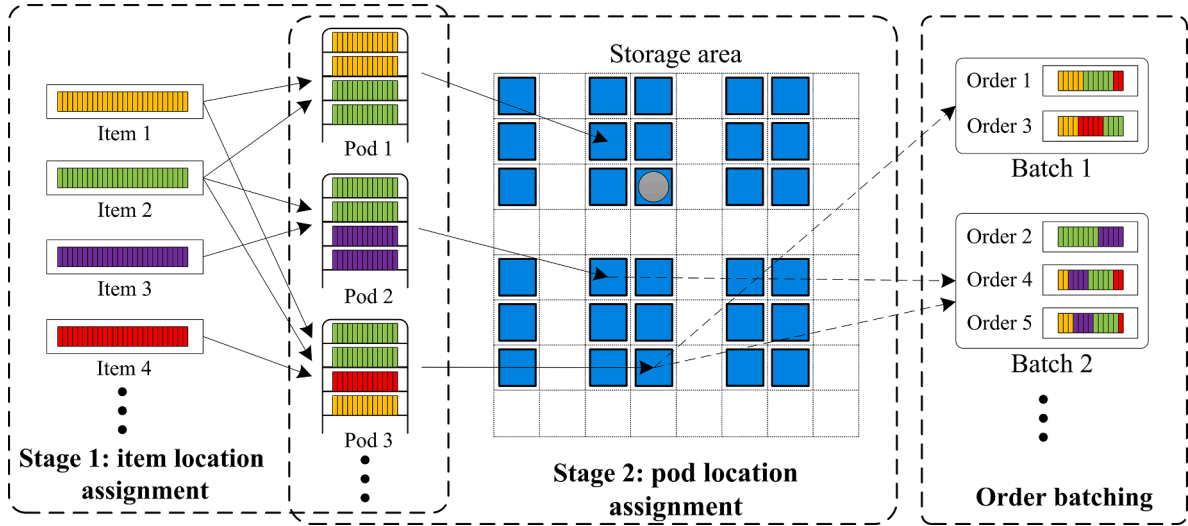


Fig. 2. Item location assignment and pod location assignment.

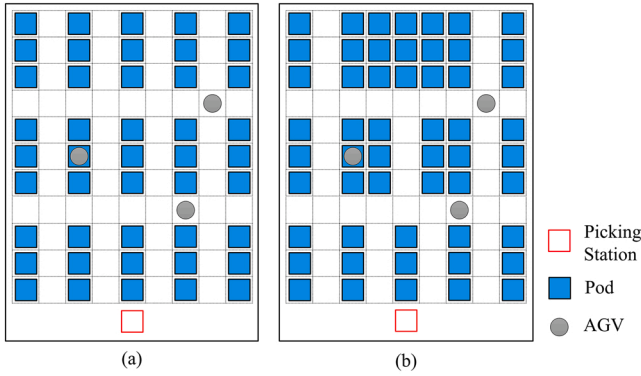


Fig. 3. Schematic diagram of RMFS with different storage depth.

and improve picking efficiency. However, handling the picking process for multiple orders simultaneously will inevitably increase the workload and pressure on the pickers. So, the number of orders in a batch is not the more the better. The number of orders in each batch is determined based on a combined value of order similarity and manual picking costs. For example, placing orders with low similarity into the same batch results in only a little reduction in the number of pod retrievals, while significantly increasing manual picking costs. This, in turn, causes a decline in the overall efficiency of the system.

The contributions of this paper are as follows:

- (i). The storage location assignment of RMFS is subdivided into item location assignment and pod location assignment and is jointly optimized with order batching. Then, a two-stage integer programming model for storage location assignment and order batching (SLAOB) is constructed with the objectives of minimizing number of pod retrieval times, order relevance, and comprehensive picking costs.
- (ii). According to the characteristics of the model, a two-stage greedy variable neighborhood simulated annealing algorithm (TGVNSA) was constructed to solve the problem.
- (iii). Use order data to guide the completion of RMFS storage location assignment, considering the dynamic storage depth and the dynamic consumption of items.
- (iv). Allow the existence of multiple depth storage areas, enabling RMFS to accommodate more pods than traditional

configurations, effectively addressing the massive order demand during peak periods.

The remaining chapters are arranged as follows: [Section 2](#) is a literature review; [Section 3](#) describes and models the joint optimization of storage assignment and order batching considering dynamic parameters; [Section 4](#) designs an algorithm to solve the model; [Section 5](#) analyzes the calculation results; and [Section 6](#) summarizes.

2. Literature review

This section summarizes relevant research and is divided into the following aspects: an overview of storage location assignment problems, an overview of order batching problems, and an overview of joint optimization problems. A summary of relevant studies about RMFS can be found in [Table 1](#).

2.1. Storage assignment

Storage assignment can be divided into item location assignment, pod location assignment, and pod location reassignment. Storage assignment is a typical generalized assignment problem. A scientific storage assignment method can shorten walking distances, reduce search times, decrease workload, and improve warehouse picking efficiency. [Hausman et al. \(1976\)](#) first proposed three basic assignment strategies: dedicated storage, random storage, and class-based storage. Subsequent literature has studied this issue from various aspects such as throughput, COI (Cube-per-Order Index), turnover rate, and the correlation between demand and structure.

Regarding the item location assignment problem, [Onal et al. \(2017\)](#) proposed a chaotic storage assignment strategy and developed a queuing model for the item assignment. The results showed that, with the increase of the chaotic level, the order picking time could be reduced by up to 16 %. [Boysen et al. \(2017\)](#) along with [Weidinger and Boysen \(2018\)](#) proposed a shared storage strategy that can significantly improve the picking efficiency of RMFS. [Lee et al. \(2020\)](#) introduced a bi-objective optimization model to maximize the relevance of SKUs within the same cluster to enhance efficiency, while also minimizing the maximum value of SKU picking frequency across different clusters to balance traffic flow. [Kim et al. \(2020\)](#) analyzed the item assignment problem in RMFS and proposed a re-optimization heuristic method to address changes in item similarity values. [J. Zhang et al. \(2023\)](#) considered the human influence in the item allocation of RMFS, developed a bi-objective mixed integer programming model aimed at maximizing the

Table 1
Summary of literature related to RMFS.

Paper	Problem	Method	Objective	Feature
Boysen et al. (2017)	ILA & OB	IP & DP	Number of rack changes & visits	Shared storage
Lamballais et al. (2017)	PLA	QN	Throughput	Storage zones
Onal et al. (2017)	ILA	QN	Makespan	Explosive storage
Weidinger and Boysen (2018)	ILA	MIP	Weighted sum of distances	Scattered storage
Weidinger et al. (2018)	PLA	MIP	Total travel distance	Interval scheduling
Xiang et al. (2018)	ILA & OB	MIP	Number of pod movements	Joint optimization
Yuan et al. (2019)	PLA	Fluid model	Total travel distance	Velocity-based storage
Lee et al. (2020)	ILA	MIP	Correlation	Traffic balanced storage
Kim et al. (2020)	ILA	MIP	Correlation	Changed similarity values
X. W. Li et al. (2020)	ILA & PLA	MIP	Correlation & distance	Turnover-rate-based decentralized storage
Keung et al. (2021)	PLA	Data-driven	Total travelling cost	Consider multi-deep costs
Keung et al. (2022)	PLA	IP	Total cost	Industrial internet of things
Wang et al. (2022)	PLA	IP	Throughput	Fishbone RMFS
Cezik et al. (2022)	PLA	Continuous variable	Total travel distance	Velocity-based storage
Mirzaei et al. (2022)	ILA & PLA	MIP	Expected travel time	Symmetry breaking constraints
J. Zhang et al. (2023)	ILA	MIP	Correlation & energy consumption	Human energy consumption
Ma et al. (2023)	ILA	MIP	Correlation	Scattered Storage
Liu and Poh (2023)	PLA	MIP	Quantity of specific items	Large items
Zarinchang et al. (2024)	ILA	Nonlinear MIP	Rack stability & picking efficiency, etc.	Worker safety
Ding et al. (2024)	PLA	IP	Total travel distance	Velocity-based
Cheng et al. (2024)	OB	SMDP & DRL	Number of pod movements	Deep reinforcement learning
This paper	ILA & PLA & OB	IP	Comprehensive picking costs	Dynamic storage depth & surplus items

Note: ILA: Item Location Assignment; PLA: Pod Location Assignment; OB: Order Batching; QN: Queueing Network; IP: Integer Programming; MIP: Mixed Integer Programming; DP: Dynamic Programming; SMDP: Semi-Markov Decision Process; DRL: deep reinforcement learning.

similarity of items in the pods and minimizing the energy consumption of pickers, then they proposed an improved knee-driven evolutionary algorithm (IKnEA) to solve the model. Ma et al. (2023) proposed a decentralized related storage strategy based on item classification and correlation to improve order-picking efficiency in RMFS. Zarinchang et al. (2024) proposed an artificial intelligence algorithm to address the storage location allocation problem in warehouses. This algorithm combines basin-hopping and simulated annealing, demonstrating its accuracy comparable to commercial solvers in real case studies.

Regarding the issue of pod location assignment, The researches by Yang, Miao, Xue, and Qin (2015); Yang, Miao, Xue, and Ye (2015); Yang

et al. (2017) on storage location assignment for AS/RS provides theoretical support for storage location assignment in RMFS. Nigam et al. (2014) used a queueing model to study the issue of random storage and nearest storage strategies in RMFS. The results showed that the random storage had higher space utilization, while the nearest storage had higher throughput. Lamballais et al. (2017) constructed a queueing network model to evaluate the performance of the RMFS. They assessed system performance using indicators such as maximum throughput, average order picking time, and robot utilization rate. The results indicate that employing a zoning storage strategy based on pod turnover rate can increase the system's maximum throughput by about 50 %. A limitation mentioned is that the study assumes each pod stores only one type of item. Weidinger et al. (2018) conducted a study on the dynamic pod location assignment in RMFS, constructed a mixed-integer programming model, and solved the model using an adaptive planning method. This reduced the total picking travel distance by 3.49 % and reduced the number of robot formations by 2.17 %. However, the theoretical model reflecting the dynamic relationship between pod storage locations was not been constructed. Yuan et al. (2019) further explored random storage, velocity-based, and class-based shelving assignment strategies, constructing a queueing model to evaluate the performance of different storage strategies. They discovered that the class-based strategy was more robust and adaptable to different warehouse configurations and layouts in RMFS. X. W. Li et al. (2020) first employed time association analysis and clustering to identify highly related products and store them on the same pod. Then they proposed a new turnover-based decentralized storage strategy (TRBDSP) to avoid AGV congestion. Additionally, they introduced an order picking performance evaluation method that considers AGV energy consumption. Keung et al. (2021) proposed a data-driven approach for regional clustering and storage location allocation to improve the operational efficiency of RMFS. Keung et al. (2022) discussed the use of the Industrial Internet of Things (IIoT) in RMFS to improve operational efficiency and developed different storage location assignment rules and strategies to minimize operating costs. It is worth noting that Keung's two papers discuss the blocking costs of pod movement in multi-deep storage areas. Wang et al. (2022) proposed a storage location assignment optimization model for fishbone RMFS, aiming to maximize operational efficiency and balance aisle workloads. Lanza et al. (2022) proposed a mixed-integer linear programming (MILP) model to allocate storage locations for product types and determine the order of allocation. The model combines effective inequalities, relaxations, and two mathematical heuristic methods to address this problem. The sorting approach in this paper can provide guidance for multi-deep storage solutions. Liu and Poh (2023) proposed decentralized storage with SKU-specific bulky locations to improve picking efficiency in response to quantity fluctuations in customer order patterns. H. R. Li et al. (2024) established a mathematical model for the storage location assignment problem in large-scale automated warehouses, considering three objectives: efficiency, pod stability, and load balancing. They proposed an improved vortex search algorithm based on attraction operation in the flow field, dimension-by-dimension dynamic radius, and leadership decision-making mechanism (FDVSA). Cezik et al. (2022); Ding et al. (2024) proposed a velocity-based pod storage location assignment (VRSLA) method for unidirectional RMFS. This method optimizes the total distance of pod movement by calculating the defined pod velocity and storage location velocity.

2.2. Order batching

The order batching problem was first proposed by Ackerman. Compared with storage assignment, order batching optimization can improve picking efficiency more significantly. The order batching problem is an NP-hard problem. Depending on whether the order information is known, order batching can be divided into online order batching and offline order batching. Generally, the offline order

batching problem is studied by heuristic algorithms, data mining, clustering, and simulation, while the online order batching problem is studied by time windows, heuristics, and other methods. Improvements to the research methods for offline order batching can also be used to study online order batching problems. Existing literature uses indicators such as maximizing batch order correlation and minimizing the number of pod moves as optimization objectives, and mainly uses queuing theory, (mixed) integer programming, clustering, and other methods to study order batching problems in RMFS.

Henn (2012) considered a dynamic online order batching problem, grouping customer orders into batches with the goal of minimizing the makespan, and adapting solutions from the offline batching problem to the online environment. Menéndez et al. (2017) proposed a multi-start variable neighborhood search method to solve the order batching problem in warehouse operations based on different strategies. Yang et al. (2020) developed an order batching optimization model for three typical storage systems and created an algorithm package that includes location distance algorithms, location selection algorithms, path planning algorithms, and order batching algorithms to address these models. Gil-Borrás et al. (2020) proposed a method that combines Greedy Randomized Adaptive Search Procedure (GRASP) and Variable Neighborhood Descent (VND) to optimize online order batching problems. The next year, Gil-Borrás et al. (2021) conducted another research on online order batching problems with multiple pickers, aiming to minimize the makespan, balance the picking time, and evenly distribute the workload among pickers. They proposed a mixed shaking variable neighborhood descent metaheuristic method to tackle this issue. Furthermore, the study also evaluated the balance of workload among pickers and the picking time. Kong et al. (2023) introduced an order batching method with forecasting and delay capabilities, considering time intervals and buyer completion rates to generate batches, aiming to minimize the total processing time of auction orders and the system response time. Gil-Borrás et al. (2024) proposed two new time window strategies and demonstrated how these time window methods affect different objective functions of the online order batching problem when the number of orders and picking stations varies. Gabellini et al. (2024) used a nonlinear machine learning-based predictive model to forecast the picking time for batch orders, then employed this prediction to guide a genetic algorithm in finding the optimal allocation of future planned batch orders to pickers, while considering each picker's individual learning and fatigue characteristics. Most of the above studies focus on manual picking, providing significant guidance for the order batching problem in RMFS. These studies indicate that using data mining methods such as clustering and association rules for order batching research in RMFS would be an effective approach. There are few studies that directly address the order batching problem in RMFS, with the most recent one being as follows. Cheng et al. (2024) introduced a semi-Markov decision process framework to simulate the Mobile Robot Order Batching Problem (MROBP), enabling continuous decision-making in dynamic warehouse environments. The proposed method leverages the adaptive learning capabilities of deep reinforcement learning to enhance order batching decisions, outperforming traditional heuristic and metaheuristic approaches.

2.3. Joint optimization

Some scholars have conducted joint optimization of the storage location assignment problem or the order batching problem in conjunction with other issues. Lin et al. (2016) developed a model under certain practical constraints to simultaneously determine the optimal order batching allocation and the shortest Manhattan path for pickers, proposing an improved particle swarm optimization algorithm to solve it. Scholz et al. (2017) proposed a model that simultaneously addresses the joint order batching, assignment, sequencing, and routing problem to minimize the total delay time of customer orders. They introduced a variable neighborhood descent algorithm that integrates various

batching, sequencing, and path planning algorithms. Boysen et al. (2017) expanded the issue into three sub-problems: order batching, order sequencing, and the sorting of pods at the picking station. The results indicate that optimized order scheduling and pod sorting can reduce the operating time of robots by at least 21 %. J. Zhang et al. (2017) studied the issue of synergistic optimization of order batching and sequencing under the premise of considering order delivery deadlines. They constructed an integer programming model for synergistic optimization, achieving an order fulfillment rate of 80.365 %. Tabrizi et al. (2023) proposed a three-stage model for optimizing online order picking and delivery in e-commerce. The first stage involves clustering products and allocating storage locations to minimize transportation and energy costs. The second stage provides a joint model for online order batching and picking path planning, considering the impact of learning effects on picking and search times to minimize completion time. The third stage applies the optimized decisions from the first two stages to the actual online order picking and delivery process, resulting in reductions of 35.65 % in total costs and 12.48 % in completion time. Momeni and Jain (2024) examined the dedicated storage policy in relation to the production planning problem and proposed a mixed-integer programming model aimed at minimizing the total costs of production and warehousing. They initially employed lexicographical constraints to ensure that storage locations were allocated adjacently to items.

It is particularly important to highlight three papers, one of which is by Xiang et al. (2018), who focused on storage assignment and order batching problems. They developed a mixed-integer linear programming model with the goal of increasing item similarity during the storage allocation phase and reducing the frequency of pod movements in the order batching phase. They designed a Variable Neighborhood Search algorithm to solve the order batching problem, and the results indicated that effective storage assignment and order batching could significantly reduce pod transportation. However, the paper did not explore the inherent correlation between storage assignment and order batching, treating them as separate phases and neglecting pod location assignment. Additionally, it assumed that the quantity of items on each pod was sufficient for orders and did not account for item depletion on the pods. The second paper is by Yang et al. (2022), who developed a Semi-Open Queueing Network (SOQN) model to describe multi-deep compact RMFS. They used Approximate Mean Value Analysis (AMVA) to solve the model and obtain performance metrics such as system throughput, robot utilization, and queue length. This is the first study to model and evaluate multi-deep compact RMFS, providing guidance for warehouse planners and managers in the design and operation of such systems. Thirdly, Mirzaei et al. (2022) addressed the storage location optimization problem in RMFS by establishing a mixed-integer programming model guided by order information. They adopted a decentralized storage allocation strategy that comprehensively considers product correlation and turnover frequency for both item assignment and pod location assignment. By introducing symmetry breaking constraints and deriving expressions for expected travel times, they simplified the complexity of the optimization model. However, the pod location assignment in their work is allocated only to a specific zone, with pods placed randomly within that zone, without considering the differences between each individual storage location.

The aforementioned literature has made significant contributions to research on storage location assignment and order batching. However, there are few studies that consider the order batching and storage location assignment for RMFS together. Exploring the intrinsic connections between storage location assignment and order batching, and conducting joint optimization, will provide good initial values for subsequent task allocation and path planning. This approach is also essential for enhancing overall efficiency, reducing costs, and improving order fulfillment capabilities in RMFS. At the same time, most literature on RMFS does not consider the possibility of multi-deep storage and typically assumes that the quantity of items on the pods adequately

meets order demands, without dynamically accounting for changes in the remaining items on the pods. If dynamic storage depth and surplus items are taken into account in RMFS research, it would enhance the system's adaptability, allowing for rapid adjustments in operational strategies in response to fluctuations in orders and diverse demands, thereby reducing the need for manual intervention and management complexity.

3. Problem description and model formulation

This section describes the joint optimization of RMFS storage assignment and order batching considering dynamic storage depth and surplus items and establishes a mathematical model.

3.1. Problem description

The storage assignment in this paper is divided into two stages: item location assignment and pod location assignment. The relationship between the two stages is illustrated in Fig. 2. Stage 1 involves optimizing item location assignment, which includes placing various types of items on the pods according to specific principles. Each pod includes multiple slots, and each slot has an upper limit on the number of items that can be stored. As order retrieval proceeds, the number of surplus items on the pods decreases accordingly. It is necessary to determine the number of pods used and the items and their quantities in each slot, so as to minimize the total number of pod retrieval times needed to fulfill order-picking tasks. Once the actual number of pods in use is determined, the system configuration, i.e. the storage depth, can be determined.

Stage 2 involves the joint optimization of pod location assignment and order batching, where each pod is positioned at an appropriate location within the system, and orders are merged for picking based on a specific principle, with the goal of minimizing comprehensive picking costs. Comprehensive picking costs include pod retrieval cost, manual picking cost, and multi-depth cost. The pod retrieval cost is determined by the location of pods and the number of pod retrieval times. The number of pod retrieval times and the results of order batching are closely interconnected. Furthermore, whether to perform order batching and the choice of different order batching strategies significantly impact the actual number of pod retrievals. The consumption of items in each slot during the order completion process is also considered. The manual picking cost aims to limit the number of orders in a batch. The more orders in a batch, the greater the number of orders workers must pick simultaneously, which increases workload, task difficulty, and error rates, resulting in higher picking costs. So, there is no need to put fewer related orders in the same batch. Multi-depth cost is used to describe the additional cost of reverse stocking when retrieving pods in the middle of multi-depth areas.

The order batching determines the number of pod retrieval times, and the number of pod retrieval times and the location of each pod jointly determine the pod retrieval cost. Moreover, the different conditions of storage location assignment can also influence the optimal solution for order batching. Therefore, there is an intrinsic coupling relationship between order batching and storage optimization, necessitating joint optimization.

Based on the above description and observations in actual production, the following assumptions are made about the system discussed in this article:

- (i). One type of item can be stored in more than one pod, and different items can be stored in one pod, but each slot can store only one type of item.
- (ii). There is an upper limit on the number of each item stored in a slot. As the retrieval progresses, the surplus items decrease.
- (iii). A pod only visits one picking point in a retrieval mission, and it returns to the original position when the picking is completed, and a pod can be retrieved multiple times.

- (iv). The number of orders in a batch cannot exceed the maximum allowed value.
- (v). The contents of the orders are known.
- (vi). The robot travel distance is calculated by Manhattan distance, and multi-depth cost is added when retrieving pods in the middle of multi-depth areas.
- (vii). The picking process for a wave order does not include mid-way replenishment operations.

The meanings of the symbols used in this paper are shown in Table 2, and the definitions and values of the decision variables are shown in Table 3.

3.2. Optimization model of item location assignment

The item location assignment problem can be described as placing items on at most m pods, each pod has Q slots, and the upper limit of the storage quantity of item i in a slot is p_i . Need to complete j orders, and each order includes multiple item requirements. The requirement is to place i type of items into pods so that the number of pod retrieval times to complete the picking task of j orders is minimized. Consider the consumption of items in each slot during the order fulfillment process, as this will affect the number of each pod retrieval times. The result is the type and quantity of items stored in each slot on each pod. This problem is essentially a bin-packing problem, and the mathematical model is as follows:

$$\min \sum_{m \in M} \sum_{j \in J} x_{mj} \quad (1)$$

$$\sum_{m \in M} y_{mji} \geq m_{ji}, \forall j \in J, i \in I \quad (2)$$

$$\sum_{j \in J} y_{mji} \leq z_{mni}, \forall m \in M, i \in I, n \in N \quad (3)$$

$$m_{ji} \cdot x_{mj} \geq y_{mji}, \forall i \in I, j \in J, m \in M \quad (4)$$

$$\sum_{i \in I} \sum_{m \in M} z_{mni} \leq C_i, \forall n \in N \quad (5)$$

$$q_{mi} p_i \geq z_{mni}, \forall m \in M, i \in I, n \in N \quad (6)$$

$$\sum_{i \in I} q_{mi} \leq Q, \forall m \in M \quad (7)$$

Table 2

Definition of symbols.

Symbol	Definition
M	Set of available pods, $M=\{1, 2, \dots, m\}$
I	Set of items, $I=\{1, 2, \dots, i\}$
J	Set of orders, $J=\{1, 2, \dots, j\}$
m_{ji}	The demand for item i in order j
C_i	The total storage volume of items i
p_i	The maximum number of items i that can be stored in a slot
Q	The number of slots on a pod
D_{xy}	Manhattan distance between (x,y) and the picking station
Z	Number of zones in the storage area.
M_1	Set of pods actually in use, $M_1=\{1, 2, \dots, m_1\}$
X	The horizontal coordinate of the storage area, $X=\{1, 2, \dots, x\}$
Y	The vertical coordinate of the storage area, $Y=\{1, 2, \dots, y\}$
K	Set of order batch, $K=\{1, 2, \dots, k\}$
P	Maximum order quantity per batch
N	Set of natural numbers
$\text{cost}_{\text{labor}}$	Manual picking cost
$\text{cost}_{\text{depth}}$	Multi-depth cost
coef_{la}	Manual picking cost coefficient
coef_{md}	Multi-depth cost coefficient

Table 3

The definition and value range of decision variables.

Decision variables	Definition and value
x_{mj}	Binary variables: 1, if pod m provides picking service for order j , otherwise, 0
y_{mji}	Non-negative integer variables: The quantity of items i picked in the pod m for order j
q_{mi}	Non-negative integer variables: The number of slots occupied by item i in the pod m
z_{mni}	Non-negative integer variables: The current quantity of item i on pod m after its n -th retrieval
p_{mxy}	Binary variables: 1, if pod m is at coordinate (x,y) , otherwise, 0
u_{jk}	Binary variables: 1, if order j is in batch k , otherwise, 0
v_{xyz}	Binary variables: 1, if point (x,y) is within the z zone, otherwise, 0
d_z	Non-negative integer variables: Storage depth of the z zone.

$$\sum_{j \in J} \sum_{m \in M} \left[\sum_{i \in I} (Q - q_{mi}) \right] \cdot x_{mj} \leq Q \quad (8)$$

$$x_{mj} \sum_{i \in I} z_{mni} - \sum_{i \in I} y_{mji} = x_{mj} \sum_{i \in I} z_{m(n+1)i}, \forall m \in M; j, i \in J; n \in N \quad (9)$$

The objective function (1) is to minimize the number of pod retrieval times during the order fulfillment process. Constraint (2) indicates that the quantity of items on the pod can satisfy the needs of all orders. Constraint (3) indicates that the quantity of items provided by each pod does not exceed its own storage capacity. Constraint (4) means that each order is served by at least one pod for picking. Constraint (5) means that the quantity of items stored does not exceed the total number. Constraint (6) indicates that the actual storage quantity of an item on each pod does not exceed the pod's maximum capacity for that item. Constraint (7) indicates that the maximum number of slots used by each pod is Q . Constraint (8) ensures that the number of empty slots on all pods that already stored items is less than Q so as to prevent excessive empty slots from wasting storage space. Constraint (9) indicates that the quantity of items stored will decrease after the pod is retrieved once, with the reduction amount being related to the order demand.

3.3. Joint optimization model of pod location assignment and order batching

Joint optimization of pod location assignment and order batching can be described as: In an area of size $X \times Y$, place non-empty pods at different locations in the system, and the Manhattan distance from each location to the picking station is known. Divide all orders into k batches according to their similarity, and pick the orders in a batch together. A batch contains at least one order and at most P orders. It requires a reasonable allocation of pod locations and order batching to minimize the comprehensive picking cost. Comprehensive picking costs include pod retrieval cost, manual picking cost, and multi-depth cost. The optimization result is the coordinates of each pod, the number of batches, and the order combinations contained in each batch. This problem is essentially a generalized assignment problem, and the mathematical model is established as follows:

$$\min \left(\sum_{m \in M_1} \sum_{j \in J} \sum_{k \in K} \sum_{x \in X} \sum_{y \in Y} D_{xy} p_{mxy} x_{mj} u_{jk} + \text{cost}_{\text{labor}} + \text{cost}_{\text{depth}} \right) \quad (10)$$

$$\sum_{x \in X} \sum_{y \in Y} p_{mxy} = 1, \forall m \in M_1 \quad (11)$$

$$\sum_{m \in M} p_{mxy} \leq 1, \forall x \in X, y \in Y \quad (12)$$

$$\sum_{k \in K} u_{jk} = 1, \forall j \in J \quad (13)$$

$$1 \leq \sum_{j \in J} u_{jk} \leq P, \forall k \in K \quad (14)$$

$$\sum_{m \in M_1} z_{mni} \geq \sum_{j \in J} \sum_{m \in M_1} u_{jk} y_{mji}, \forall i \in I, k \in K, n \in N \quad (15)$$

$$x_{mj} \sum_{i \in I} z_{mni} - \sum_{i \in I} y_{mji} = x_{mj} \sum_{i \in I} z_{m(n+1)i}, \forall m \in M; j, i \in J; n \in N \quad (16)$$

$$\sum_{z=1}^Z \left[d_z \cdot \left\lfloor \frac{X}{d_z + 1} \right\rfloor + \left(X - \left\lfloor \frac{X}{d_z + 1} \right\rfloor \cdot d_z - 1 \right) \right] \cdot \frac{Y - Z - 1}{Z} \leq \text{card}(M_1) \quad (17)$$

$$\text{cost}_{\text{labor}} = \sum_{la=1}^P \left[\text{coef}_{la} \cdot \sum_{k \in K} \left(\sum_{j \in J} u_{jk} = la \right) \right] \quad (18)$$

$$\text{cost}_{\text{deep}} = \sum_{x \in X} \sum_{y \in Y} \text{coef}_{md} \cdot D_{xy} \cdot d_z \cdot v_{xyz} \quad (19)$$

The objective function (10) is to minimize the comprehensive picking costs when finishing the order fulfillment process. Constraint (11) means that a pod can only occupy one position. Constraint (12) indicates that at most one pod can be placed at each position in the storage range. Constraint (13) means that each order is assigned to a batch. Constraint (14) means that each batch contains at least one order and at most P orders. Constraint (15) indicates that the required items in each batch do not exceed the total storage capacity in the pod. Constraint (16) indicates that the quantity of items stored will decrease after the pod is retrieved once, with the reduction amount being related to the order demand. Constraint (17) indicates that the setting of depth for each zone must meet the requirements for pod placement. Constraint (18) represents the manual picking cost. Constraint (19) represents the multiple depth cost.

3.4. Problem analysis

Existing literature has established that both the storage location assignment problem and order batching are NP-hard (Ding et al. 2024; Yang et al. 2020). The joint optimization of these two fundamental problems, as discussed in this paper, also exhibits NP-hard computational complexity. Furthermore, the decision variables in this study include both binary (0–1) and integer variables, which adds to the difficulty of solving the proposed model. According to Mirzaei et al. (2022), using the GUROBI solver for pod location assignment tasks involving 100 orders and 50 item types requires approximately one day of computation. In practice, the actual number of orders and items is significantly larger, making it challenging for exact algorithms to find solutions within a reasonable time frame.

Moreover, this paper not only addresses pod location assignment but also incorporates item placement and order batching. Each order requires multiple items, establishing a connection between order demand and the surplus items on the pods through the variable z_{mni} . Additionally, different storage depths can be set in various zones, further enlarging the solution space compared to addressing each problem in isolation. As the number of pods, storage locations, item types, and order quantities increases, the solution space grows exponentially. As a result, it is difficult to achieve solutions with exact algorithms within a limited time, highlighting the need for a fast and effective heuristic algorithm to address the problem.

4. Algorithm design

The joint optimization of storage assignment and order batching in RMFS with dynamic storage depth and surplus items discussed in this paper is a multi-stage combinatorial optimization problem. There is an obvious coupling relationship between each stage of this problem. However, the solution space of this problem is vast and there are so

many constraints. Obviously, it is difficult to find the optimal value using an accurate algorithm. Since there is a clear optimization direction at each stage of this problem, it is suitable to use heuristic algorithms to solve it, especially neighborhood search algorithms. The mathematical model constructed in Section 3 may not achieve satisfactory optimization results using a single algorithm; therefore, it is necessary to design an algorithm that converges quickly, has high precision, and possesses a certain degree of adaptability.

4.1. The framework of TGVNSA

The simulated annealing algorithm mimics the annealing process in physics by probabilistically accepting poorer solutions to avoid getting trapped in local optima, making it a global optimization algorithm (Kirkpatrick et al. 1983). Meanwhile, variable neighborhood search dynamically alters the neighborhood structure during the search process, exploring the solution space through various structures and selecting an appropriate one based on the current solution. This

approach exhibits adaptability and the ability to escape local optima (Hansen and Mladenovic 2001). Due to the vast solution space of the model presented in this paper, it is easy to get trapped in local optima; therefore, a global optimization algorithm is essential for effectively finding the optimal solution. Combining simulated annealing with variable neighborhood search is a reasonable choice, and incorporating greedy search into various neighborhood structures can both accelerate the search process and ensure solution quality. Greedy search selects the locally optimal choice at each step based on the current state, without considering the subsequent global implications. While it can often quickly find a feasible solution, it does not always guarantee a global optimal solution. In summary, the algorithm design in this paper is based on the simulated annealing framework, incorporating variable neighborhood search and greedy search to develop a hybrid algorithm that achieves rapid convergence while guaranteeing global optimality. This algorithm is named the Two-stage Greedy Variable Neighborhood Simulated Annealing Algorithm (TGVNSA).

The flowchart of TGVNSA solving the joint optimization of storage

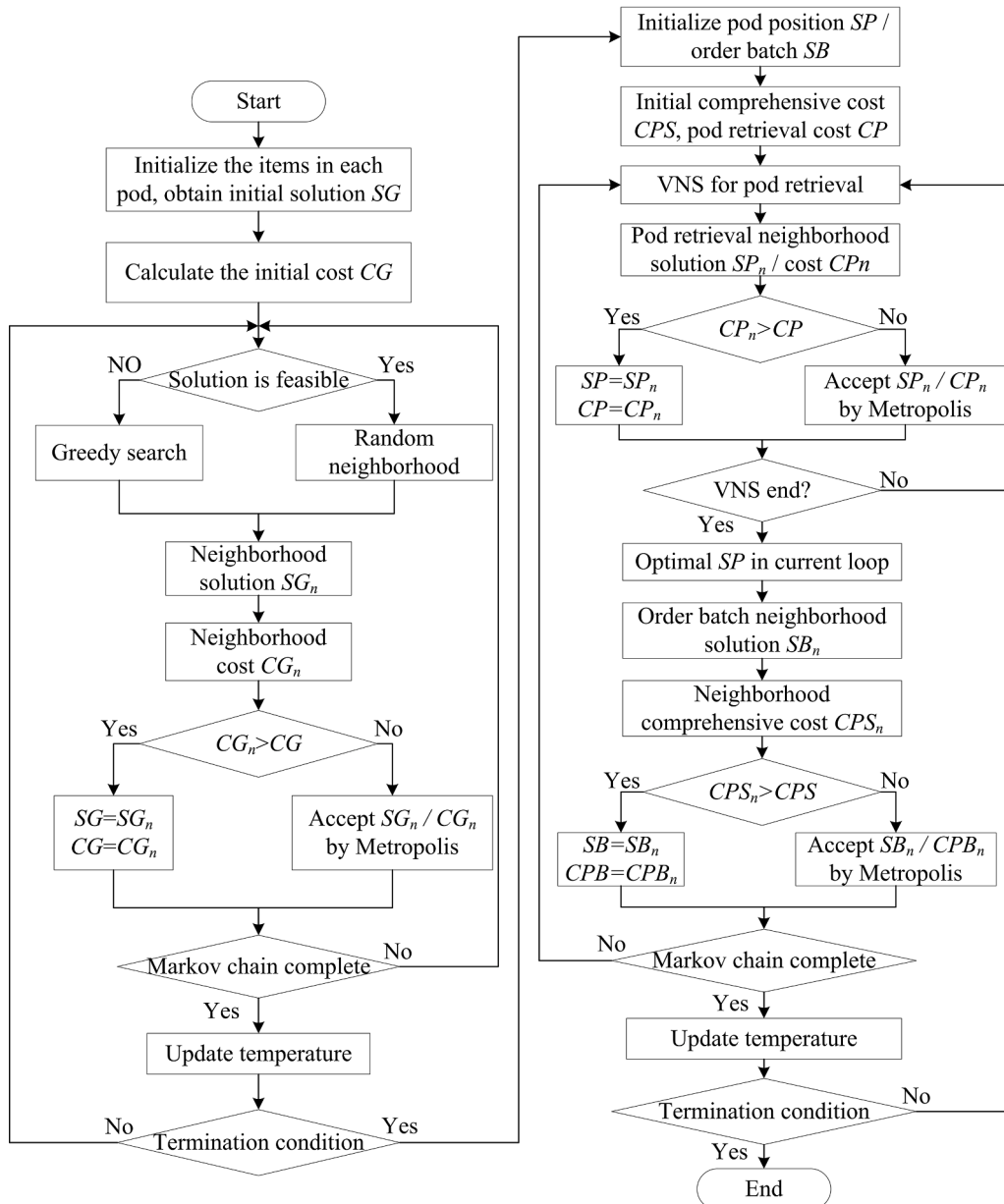


Fig. 4. Flow chart of TGVNSA.

assignment and order batching in RFMS with dynamic storage depth and surplus items is shown in Fig. 4.

The first stage of the algorithm completes the optimization of item location assignment. The first step is to initialize the types and quantities of items in each pod. The initialization algorithm is shown in Section 4.3. After obtaining the initial cost, the first stage of the greedy variable neighborhood simulated annealing process is performed. At each temperature, a Markov chain of a certain length is constructed to obtain the optimal solution at that temperature. The process is as follows: first, determine whether the current solution meets the order requirements. If it does not, the greedy algorithm is used to systematically increase the missing items. If the order requirement is met, the type or quantity of items in the pod is changed through a random neighborhood. Then, compare the neighborhood solution with the current solution. If the neighborhood solution is better, accept the neighborhood solution. If the current solution is better, accept the neighborhood solution according to the Metropolis criterion. Until the length of the Markov chain reaches a specified value, it is considered that the optimal solution at this temperature has been found. Subsequently, the temperature is updated, and the next Markov chain will be created to seek the optimal value at the next temperature. The temperature continuously decreases as the algorithm iterates a set number of times until the termination condition is met. The flowchart for the first stage is shown on the left side of Fig. 4.

After the item location assignment optimization, the item combination in each pod is obtained to initialize the pod locations and order batches. The initialization algorithm is shown in Section 4.3. Then the second stage of greedy variable neighborhood simulated annealing process is started to perform joint optimization of pod location assignment and order batching. The process of constructing the Markov chain at a certain temperature in this stage is as follows. Given the current pod location solution, order batching solution, pod location costs, and total costs, it is necessary to find the neighborhood solutions for both the pod location and order batching. Considering that the neighborhood of the pod location is much larger than the neighborhood of the order batch, in order to converge as quickly as possible, a variable neighborhood search using the Metropolis criterion is performed in each iteration of the Markov chain, with the aim of obtaining the optimal neighborhood solution for the pod location at current temperature. Then, neighborhood searches are conducted on the order batching. Similarly, accepting the solutions based on the comprehensive costs and the Metropolis criterion. Until the length of the Markov chain reaches the specified value, it is considered that the optimal pod location solution and order batching solution for the current temperature have been found. Then the temperature is updated and iterated a certain number of times until the termination condition is reached. The flowchart for the second stage is shown on the right side of Fig. 4.

4.2. Encoding

This paper adopts real number coding and uses a two-dimensional array to describe the current state of the pod, which contains information on the type of items, the number of items, and the pod location. An example of pod encoding is shown in Fig. 5. In this example, each row in the array represents information about a pod, it is assumed that each pod has 6 slots, so the first 6 numbers in each row represent the type of item, the middle 6 numbers represent the number of items, and the last two numbers represent the coordinates of the pod in the system.

Similarly, a two-dimensional array is used to encode the order batches. An example of the encoding method is shown in Fig. 6. In this example, each row represents a combination of orders included in a batch, and the number represents the order number.

4.3. Initial solution generation

The TGVNSA proposed in this paper does not depend on the quality of the initial solution, and the initial solution at each stage is generally based on the random principle. The initial solution for item location assignment is constructed through Algorithm 1. Each item is placed in a slot on a random pod. One type of item is allowed to occupy multiple pods and multiple slots. The number of items in each slot does not exceed the maximum limit. The initial solution constructed in this way has sufficient randomness and also indicates that some items are originally stored in the pods, which is more in line with the actual situation.

Algorithm 1 Constructing the initial solution for item location assignment

Input: M, I, p_i, Q

Output: $types$ (types of items on each pod), $quantity$ (quantity of items on each pod)

(continued on next page)

Batch 1	7	28	65	33	19	52
Batch 2	43	21	39	20		
Batch 3	6	43				
⋮						
Batch k	10	62	1	49	55	

Fig. 6. Encoding method of batches.

	Type of items	Quantity of items	Coordinates
Pod 1	12 15 9 45 27 50	2 6 8 3 7 8	1 3
Pod 2	33 42 5 19 24 8	7 5 8 2 1 4	1 5
⋮	⋮	⋮	⋮
Pod m	3 16 37 46 26 4	1 8 8 7 3 5	6 9

Fig. 5. Encoding method of pods.

(continued)

Algorithm 1 Constructing the initial solution for item location assignment

```

1: types = zero(length(M), Q);
2: quantity = zero(length(M), Q);
3: for i = 1: length(M)
4:   types[i,:] = randperm(length(J), min(length(J), Q));
5:   for j = 1: Q
6:     n = types[i, j];
7:     quantity[i, j] = randperm(pn, Q);
8:   end
9: end
10: return types and quantity;

```

The initial solution for pod location assignment and order batching is constructed through Algorithm 2. The pods are randomly arranged to storage locations in the system until all pods are assigned locations. The initial solution of order batching is to place only one order in each batch, which is the worst solution in order batching. TGVNSA is also applicable to such initial solutions.

Algorithm 2 Constructing the initial solution for pod location assignment and order batching

Input: $M_1, J, K, location$ (system configuration)
Output: $position$ (location of each pod), $batch$ (combinations of orders in each batch)

```

1: position = zeros(length(M1), 2);
2: index = 1;
3: for i = 1: size(location, 1)
4:   for j = 1: size(location, 2)
5:     if point(i, j) is storage location
6:       position(index,:) = [j, i];
7:       index = index + 1;
8:     Check whether each pod has been assigned coordinates, if so, break;
9:   end
10: end
11: end
12: batch = zeros(length(J), K);
13: numbers = randperm(length(J));
14: batch(:, 1) = numbers;
15: return position and batch;

```

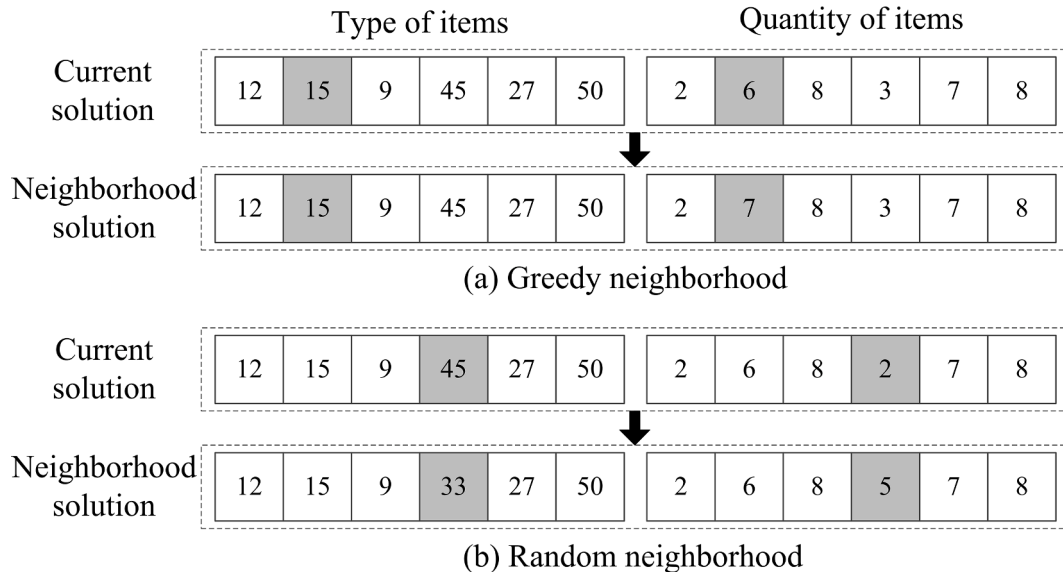
4.4. Neighborhood solution generation

The neighborhood has a direct impact on the algorithm's exploration ability and convergence. The TGVNSA proposed in this paper adopts a

variety of neighborhoods. In order to meet the order requirements during the item location assignment, two different neighborhoods are used: one based on greedy search and the other based on random principle. Before creating a neighborhood solution, check whether the current solution can meet the order requirements. If not, use the neighborhood based on a greedy search to increase the missing items in the pods. If the order requirements are met, a neighborhood based on the random principle is used. The two neighborhood structures are shown in Fig. 7. Fig. 7(a) shows a greedy neighborhood. When it is detected that the number of items with sequence number 15 is insufficient during the order retrieval process, the number of items with sequence number 15 is increased in the neighborhood solution. Fig. 7(b) shows a neighborhood based on the random principle. We select a slot of a pod and change the type and quantity of items stored in it. In order to increase the speed of the algorithm, in the random neighborhood, the quantity of items is changed to any value between the current value and the maximum value.

In the joint optimization of pod location assignment and order batching, the neighborhood range of pod location is much larger than the neighborhood range of order batching. For pod location assignment, convergence is too slow if a single neighborhood is used. In order to converge as quickly as possible, a variable neighborhood search is performed in each iteration of the Markov chain. Since the temperature information is already available, in the variable neighborhood search, the Metropolis criterion is also used to accept poor neighborhood solutions, with the goal of obtaining the optimal neighborhood solution for the pod location at the current temperature. Pod location assignment uses three different neighborhood structures: 9-grid optimal neighborhood, column optimal neighborhood, and random neighborhood. Fig. 8 (a) shows the 9-grid optimal neighborhood. The first number in the square is the serial number of a pod, and the number in the bracket is the number of retrieval times for this pod. This neighborhood is to arrange the pods in ascending order according to the number of retrieval times within the 9 grids. Fig. 8(b) shows the column optimal neighborhood, and this neighborhood is to arrange the pods in ascending order according to the number of retrieval times within a column. The random neighborhood is to swap any two pod locations in the system (not shown in the figure).

The greedy neighborhood is suitable for order batching. The operation is as follows: select order A in any batch m in the current order batching solution, and find the order B with the greatest similarity to A in other batches. If batch m is not full, add order B to it. As shown in

**Fig. 7.** Neighborhood structures of item location assignment.

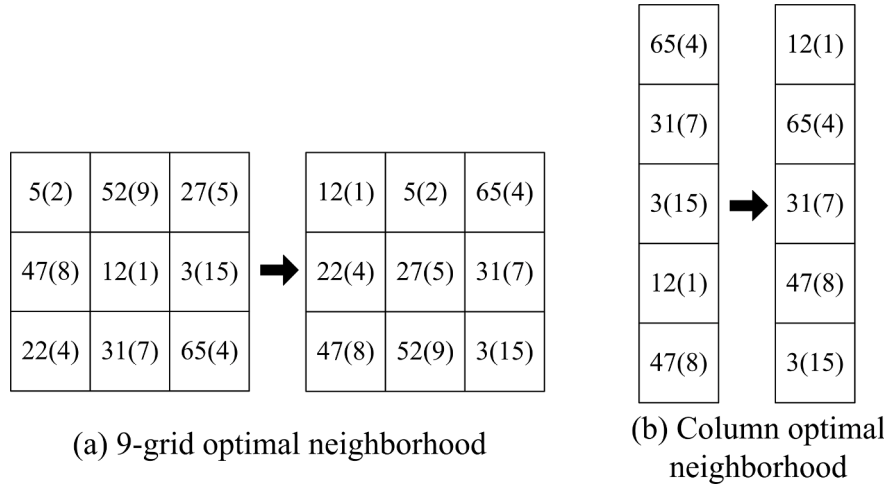


Fig. 8. Neighborhood structures of pod location assignment.

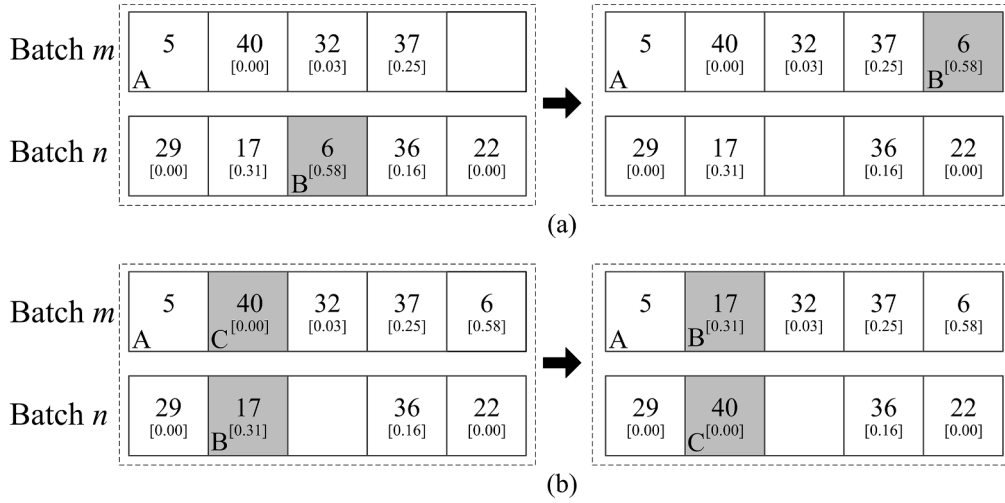


Fig. 9. Neighborhood structures of order batching.

Fig. 9(a), the upper number in the square represents the serial number of an order, and the number in the square brackets represents the similarity with order A. If batch m is full, find order C in batch m which has the lowest similarity to order A. If the similarity value of B is greater than C, swap B and C, otherwise skip, as shown in Fig. 9(b).

Jaccard similarity is a fast and efficient metric used to calculate the similarity between two sets. Since the basic Jaccard similarity may not accurately describe the similarity between sets containing duplicate elements, this paper proposes a Weighted Jaccard Similarity Calculation Algorithm to characterize the similarity between orders. The pseudo-code is presented in Algorithm 3.

Algorithm 3 Calculate order similarity

Input: order A, order B

Output: similarity (similarity between orders A and B)

```

1: jaccard_similarity = 0;
2: unique_elements = union(A, B); % Find all unique elements in A and B.
3: count_A = histc(A, unique_elements); % Frequency of each unique element in A
4: count_B = histc(B, unique_elements); % Frequency of each unique element in B
5: intersection = sum(min(count_A, count_B)); % Intersection (use minimum frequency)
6: union_count = sum(max(count_A, count_B)); % Union (use maximum frequency)
7: jaccard_similarity = intersection / union_count; % Calculate the Jaccard similarity
8: return jaccard_similarity;

```

4.5. Energy function calculation

The energy function of the TGVNSA is calculated according to the objective functions of each stage in Section 3. Each stage of this algorithm needs to calculate the number of pod retrieval times. It is necessary to consider the consumption of the number of items in each pod during the order retrieval process, which is a practical approach. Algorithm 4 is the calculation process of the number of pod retrieval times considering dynamic surplus items.

Algorithm 4 Calculate the number of pod retrieval times considering dynamic surplus items

Input: types(types of items on each pod), quantity(quantity of items on each pod), orders(item requirements in the order)

Output: cost(Costs associated with the number of pod retrieval times)

```

1: calls = 0; % Record the number of pod retrieval times
2: for i = 1:size(orders, 1) % Traverse all orders
3:   row_orders = orders(i,:); % Extract a required item from the order
4:   for j = 1:length(types) % Traverse all pods
5:     id = find(ismember(types(j,:), row_orders));
6:     if any(id) && any(quantity(j, id)) % The pod contains this item
7:       calls = calls + 1; % The pod is called

```

(continued on next page)

(continued)

Algorithm 4 Calculate the number of pod retrieval times considering dynamic surplus items

```

8:      for  $n = 1:\text{length}(id)$  % Find the slot where the item is located in the
      pod
9:          need_product = types(j, id(n));
10:         count = numel(find(row_order == need_product)); % Item demand
11:         if the number of items on the pod is sufficient for order  $i$ 
12:             quantity(j, id(n)) = quantity(j, id(n)) - count; % Reduce surplus
      items
13:         row_order(indices(1:count)) = -1; % Mark this item as retrieved
14:         else % the number of items on the pod is not sufficient for order  $i$ 
15:             row_order(indices(1: quantity(j, id(n)))) = -1; % Part of items
      retrieved
16:         quantity(j, id(n)) = 0; % Set the surplus items on this pod to 0
17:         end
18:         orders(i,:) = row_order; % Update orders
19:         end
20:     end
21: end
22: end
23: aa = sum(sum(new_orders > 0));
24: cost = calls + (10 * aa); % Add non-completion penalty
25: return cost;

```

In the joint optimization of pod location assignment and order batching, the energy function is the comprehensive picking cost in the current state, which is composed of pod retrieval cost, manual picking cost, and multi-depth cost. Combined with the cost function in Section 3.3, the energy function reflecting the comprehensive picking cost is calculated by Algorithm 5. The pod retrieval cost is equal to the product of the number of pod retrieval times and the distance from the pod to the picking station. The number of pod retrieval times is also calculated by Algorithm 4.

Algorithm 5 Calculate the comprehensive picking cost

Input: types(types of items on each pod), quantity(quantity of items on each pod), orders(item requirements in the order), batch(combinations of orders in each batch), position(location of each pod), IO_point(picking station location)

Output: cost(comprehensive picking cost)

```

1:  batch_order_list records the items required for all orders in each batch;
2:  actual_calls records the number of pod retrieval times calculated by
   Algorithm 4;
3:  distances = abs(position - IO_point);
4:  manhattan_distances = sum(distances, 2);
5:  result = manhattan_distances * actual_calls;
6:  retrieval_cost = sum(result(:));
7:  for  $i = 1:\text{size}(\text{batch}, 1)$ 
8:      if the number of elements in batch( $i$ ,:) is 3
9:          num3 = num3 + 1;
10:     if the number of elements in batch( $i$ ,:) is 4
11:         num4 = num4 + 1;
12:     if the number of elements in batch( $i$ ,:) is 5
13:         num5 = num5 + 1;
14:     end
15:     manual_cost = length(position) * (coef3 * num3 + coef4 * num4 + coef5 *
num5);
16:     mdepth records the distance between each coordinate and the picking
station;
17:     mdepth_d records the depth of each pod;
18:     mdepth_cost = coef_md * (mdepth * mdepth_d);
19:     cost = retrieval_cost + manual_cost + mdepth_cost;
20:     return cost;

```

5. Computational experiments

This section will analyze the results of solving the joint optimization of storage assignment and order batching in RMFS with dynamic storage depth and surplus items through TGVNSA, and verify the performance of the algorithm.

5.1. Calculation results

The most prominent advantage of considering dynamic storage depth in RMFS compared to traditional systems is that the configuration can be flexibly changed according to order requirements in the item location assignment stage. The main manifestations are allowing the number of pods to exceed the maximum quantity limit of traditional systems, and adopting different storage depths for high-frequency and low-frequency pods to improve system efficiency. This section sets different parameters and order requirements for systems of different sizes and solves them through the TGVNSA. The calculation process is written in MATLAB R2018b and tested by running the program on a device with Intel(R) Core(TM) i7-7500U CPU 2.70 GHz processor and 8G RAM.

The data in the experimental cases is derived from a material supermarket and semi-finished product warehouse of a manufacturing company in eastern China. Three warehouses of different sizes were selected for the experiments. The size of the small warehouse is 10×10 (with side lengths of the cargo space and the aisle both equal to 1); the medium warehouse is 20×20 ; and the large warehouse is 30×30 . The parameters for each warehouse are shown in Table 4, where the upper limit of types of items is set to 96 for the small warehouse, 220 for the medium warehouse, and 380 for the large warehouse. Each warehouse contains three configurations: Configuration I uses the traditional single-depth storage, as shown in

Fig. 10(a). Configuration II uses a combination of single-depth and double-depth storage, as shown in Fig. 10(b). Configuration III adopts single-depth storage in the high-frequency zone (zone 3), double-depth storage in the middle zone (zone 2), and multi-depth storage in the low-frequency zone (zone 1), as shown in Fig. 10(c). The picking station in the system is located in the middle of the bottom.

In this section, the order demand is set as follows: 20 % of the items have a higher demand, which can satisfy 80 % of the order content. The relevant parameter settings in the example are shown in Table 5. The number of iterations is set to 100, 200, and 500 for the small, medium, and large warehouses, respectively, since larger systems require more iterations to achieve an optimal solution. The manual picking cost coefficient $coef_{ia}$, is assigned values of 1, 1.2, 1.4, and 1.8, corresponding to order quantities of 1, 2, 3, 4, and 5 in a single batch, this data is approximately obtained from the results regarding personnel energy consumption at picking stations from the literature by J. Zhang et al. (2023). The multi-depth cost coefficient $coef_{md}$, takes values of 1, 1.2, 1.5, and 2, corresponding to storage depths of 1, 2, 3, and 4, respectively, this data is approximately obtained from the picking costs for multi-depth storage as reported by Lanza et al. (2022) and Yang et al. (2022).

The results of the corresponding examples are shown in Table 6.

According to Table 6, considering the dynamic storage depth in RMFS shows outstanding advantages when dealing with orders with different demands: it can meet the order requirements that are close to or exceed the system's saturation capacity. In case C1, the joint optimization of storage assignment and order batching is carried out in a 10×10 system using configuration I, so that it could meet the picking task of completing 30 orders (requiring 1,200 items) at one time. The heat map of the number of pod retrieval times for case C1 is shown in Fig. 11

Table 4
The parameters of the warehouse.

Parameter	Value
Types of items	96/220/380
Quantity of items in each order	40
Demand skewness	20/80
Maximum capacity of items in a slot	8(90 %)/6(10 %)
Q	6
P	5

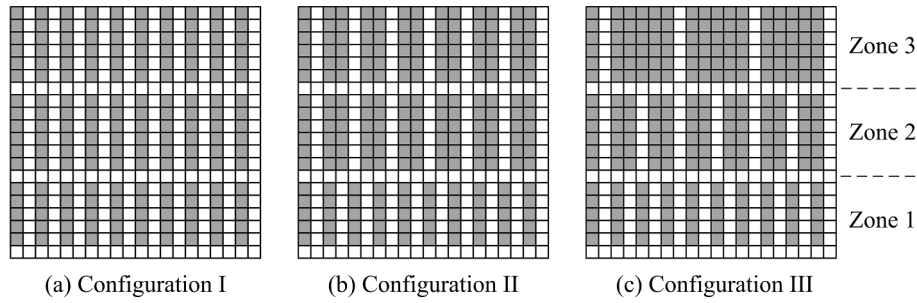


Fig. 10. System configuration diagram.

Table 5
Parameters for the TGVNSA algorithm.

Parameter	Value
The initial temperature	1000
Cooling coefficient	0.95
Markov chain length	50
Iterations	100/200/500
Manual picking cost coefficient $coef_{ia}$	1/1/1.2/1.4/1.8
Multi-depth cost coefficient $coef_{md}$	1/1.2/1.5/2

Table 6
Results of different cases.

Case	Size	Cfg	MNP	QO	ANPU	NPRT	NB	CPC
C1	10 × 10	I	40	30	40	170	7	3412
C2	10 × 10	I	40	50	—	—	—	—
C3	10 × 10	II	48	50	48	352	12	6327
C4	10 × 10	III	56	50	50	339	11	6764
C5	20 × 20	I	170	130	170	1725	28	50,725
C6	20 × 20	II	206	130	182	1651	27	47,836
C7	20 × 20	II	206	200	—	—	—	—
C8	20 × 20	III	224	200	224	2751	45	91,692
C9	30 × 30	I	390	300	390	5269	71	227,303
C10	30 × 30	II	460	400	460	6982	109	307,282
C11	30 × 30	III	525	500	525	8502	135	410,371

NOTE: Cfg: Configuration, MNP: Maximum number of pods, QO: Quantity of order, ANPU: Actual number of pods in use, NPRT: Number of pod retrieval times, NB: Number of batches, CPC: Comprehensive picking costs.

(a). In case C2, the 10 × 10 system in the configuration I cannot meet the item location assignment requirements of 50 orders (requiring 2,000 items) because there are insufficient pods in the system. In case C3, the system is modified to configuration II, where some areas adopted double-depth storage, increasing the maximum number of pods accommodated in the system. The results showed that it could complete 50 orders. The heat map of case C3 is shown in Fig. 11(b). In case C4, the system is changed to configuration III, which involves multi-depth storage in some areas, further increasing the maximum number of pods that can be accommodated. The results showed that the frequency of pod usage was close to C3, but it incurred a higher comprehensive picking cost. This is because the multi-depth storage areas require additional costs for retrieval. The heat map of case C4 is shown in Fig. 11(c). However, the configuration III used in the C4 can obviously support more order retrieval tasks.

Comparing C5 and C6, it can be seen that under the same conditions, different outcomes are obtained in different configurations. For the same task of completing 130 orders (requiring 5,200 items), configuration II, which adopts double-depth storage, enables a denser placement of racks, ultimately resulting in better performance than configuration I. The heat map of case C5 and case C6 are shown in Fig. 11(d) and (e). However, when the order demand increases to 200 (requiring 8,000 items), case C7 shows that the system using configuration II cannot complete the assignment task for so many orders. Case C8 indicates that configuration

III can meet the needs of 200 orders. The heat map of case C8 is shown in Fig. 11(f).

In cases C9, C10, and C11, with a larger number of pods and 380 types of items, the highest order quantity reached 500 (requiring 20,000 items), verifying that the proposed algorithm still has considerable effects in large-scale systems. Heat maps about cases C9, C10, and C11 are shown in Fig. 11(g), (h) and (i).

The heatmap in Fig. 11 illustrates the pod retrieval times at various locations in the storage area that are accessed during the completion of all orders. The colored squares in the figure represent the pods, with colors closer to yellow indicating a higher number of retrieval times, while colors closer to blue indicate fewer retrievals. The bar chart on the right displays the number of accesses corresponding to each color. From the heatmap, it is visually apparent that pods with a higher retrieval frequency are located closer to the picking station, while those with a lower retrieval frequency are farther away. Additionally, the pods placed on the inner side of the multi-depth area have the least number of retrieval times. This figure demonstrates that the model and algorithm proposed in this paper yield reasonable results.

5.2. Parameter sensitivity analysis

This section conducts parameter sensitivity analysis on the proposed system and algorithm. The parameters considered include item demand skewness, item types, maximum order quantity per batch, and average storage depth (system configuration).

First, we analyze the impact of different item demand skewness on system operational results. Using the basic data of case C4, calculate the number of pod retrieval times (NPRT) and the comprehensive picking costs (CPC) under the demand skewness of 20/80, 40/80, and 60/80, where 20/80 indicates that 20 % of the items satisfy 80 % of the order demand. To avoid chance, each data is averaged from the results of 10 repeated runs. The results are shown in Fig. 12(a). It can be seen that under the same conditions, the fewer the high-demand items, the fewer the number of pod retrieval times, and the lower the comprehensive picking costs. Conversely, the more the item demand skewness tends to average, the greater the number of pod retrieval times and the higher the comprehensive picking costs.

Upon further analysis of the impact of the different number of item types on the system operational results, calculations are performed for cases with 50, 100, and 150 item types, based on case C4, as shown in Fig. 12(b). It is evident that, under the same conditions, the greater the number of item types, the more frequent the pod retrieval instances, and the higher the comprehensive picking costs. This means that orders of multiple varieties and small batches often require more pods to meet the demand than orders of fewer varieties and large batches. The system proposed in this paper, which takes into account the dynamic storage depth and surplus items, has a stronger ability to cope with the demand for multiple varieties and small batches.

The impact of the maximum order quantity per batch on the system operation results is shown in Fig. 12(c). The accumulation of manual picking costs is also taken into account in the calculation process. It can

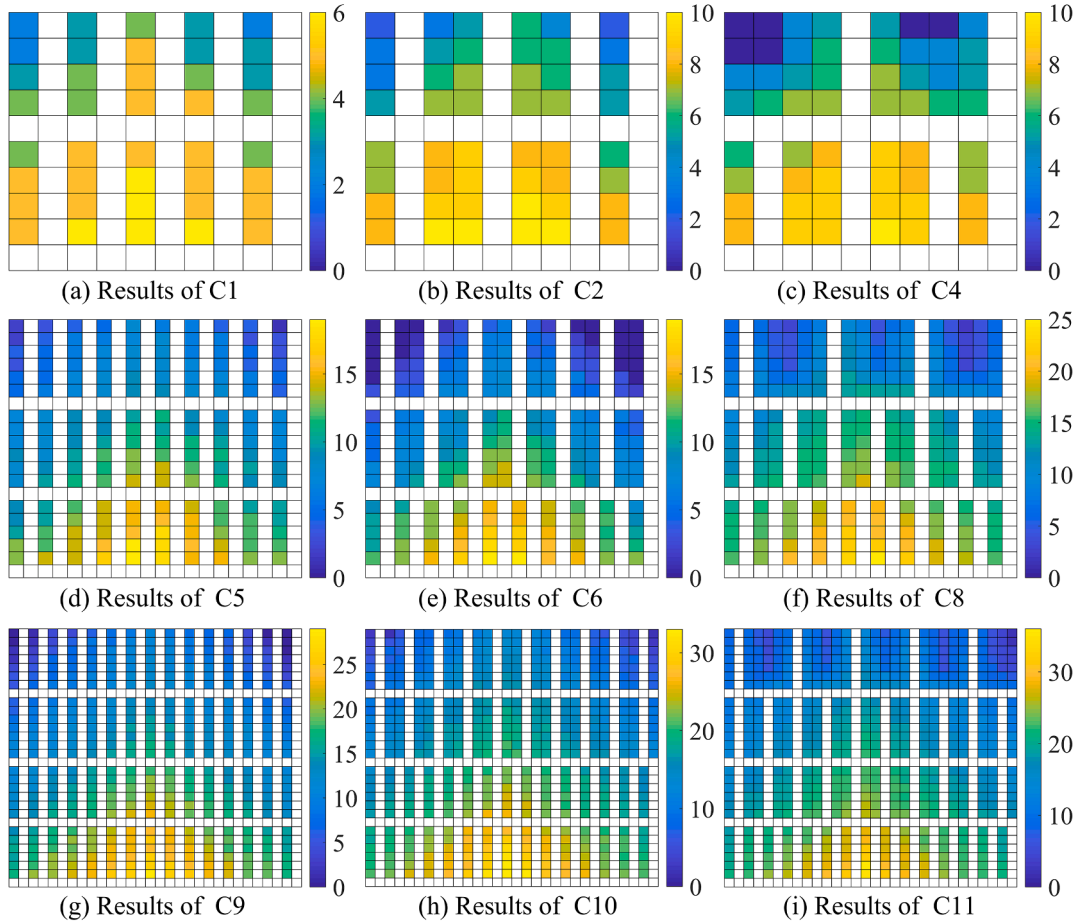


Fig. 11. The heat map of the number of pod retrieval times.

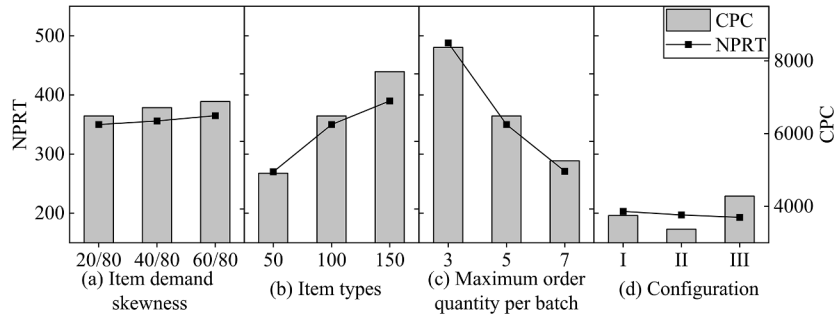


Fig. 12. Parameter sensitivity analysis.

be concluded that the larger the maximum number of orders allowed in each batch, the less frequently pods need to be retrieved, and the comprehensive picking costs are subsequently reduced. In practice, if the maximum order quantity per batch is set too high, it could overwhelm manual picking operations, increase error rates, and cause congestion with pods and robots piling up near the picking station. Therefore, setting a predetermined limit for the maximum order quantity per batch before the system is operational is crucial. This article serves as a foundation for establishing this limit.

Different system configurations, that is, different storage depths, will affect the system operation results. The sensitivity of this item needs to be verified. In order to compare the three configurations, a 10×10 system is selected to complete the retrieval task of 40 orders. The results are shown in Fig. 12(d).

Average storage depth (system configurations) will affect the system operation results. The sensitivity of this needs to be verified. In order to compare the three configurations, a 10×10 system is selected to complete the retrieval task of 40 orders. The results are shown in Fig. 12 (d). It can be seen that under the same circumstances, adopting a configuration with a high storage depth will reduce the number of pod retrieval times, because it provides more pods for use. However, when the storage depth is greater than 2, there is an additional multi-depth cost, which increases the comprehensive picking costs.

It can also be seen from Fig. 12 that changes in item types and maximum order quantity per batch have a greater impact on the number of pod retrieval times and the comprehensive picking costs, which is far greater than the impact of changes in other parameters.

5.3. Comparison of different algorithms

In this section, the proposed TGVNSA algorithm is compared with other algorithms to verify its performance. In the two categories of algorithms, namely, neighborhood search and population optimization, several algorithms are selected to be applied to solve the problem proposed in this article. The comparative algorithms include the Random Neighborhood Simulated Annealing (RNSA), Greedy Variable Neighborhood Search (GVNS), and Improving Adaptive Genetic Algorithm (IAGA).

The initial solution generation in the comparison algorithms follows the method outlined in Section 4.3 of this paper, while the calculation of the fitness function uses the approach described in Section 4.5. In the RNSA algorithm, random neighborhoods are employed for constructing the Markov chain. The GVNS algorithm utilizes both random variable neighborhoods and greedy neighborhoods, and the design of the neighborhood structures in the GVNS algorithm is the same as that in the TGVNSA algorithm. The IAGA dynamically adjusts the crossover and mutation probabilities to better balance global search and local exploration, enhancing the convergence speed of the algorithm. The calculation methods for crossover and mutation probabilities refer to Y. Li et al. (2022). The parameter values for the three comparison algorithms are shown in Table 7.

Different algorithms are used to calculate the feasible cases in Section 5.1, and the comparison of the following objective values is made: number of pod retrieval times, comprehensive picking costs, CPU time, and the comprehensive quality in the item location assignment (CQ). Among them, the comparison of the number of pod retrieval times, comprehensive picking costs, and CPU time is calculated by the formula $\text{VALUE}(\text{other})/\text{VALUE}(\text{TGVNSA}) \times 100\%$. The larger the value, the worse the comparative algorithm performs.

CQ represents the effect of the first stage of the system, which is calculated by formula (20). The parameters involved in determining CQ are the actual number of pod retrieval times AN , the variance of the number of pod retrieval times VN , and the number of unused slots NN . The higher the CQ value, the worse the performance of the comparative algorithm, indicating that the ability to cope with order fluctuations after item location assignment is very weak. The purpose of this comparison is to clearly understand the key metrics during the operation of each algorithm, allowing for a more reasonable evaluation of each one.

$$CQ = \frac{1}{3} \left(\frac{AN(\text{other})}{AN(\text{TGVNSA})} + \frac{VN(\text{other})}{VN(\text{TGVNSA})} + \frac{NN(\text{other})}{NN(\text{TGVNSA})} \right) \times 100\% \quad (20)$$

The comparison results are shown in

Fig. 13. It can be seen that TGVNSA has good performance in solution quality and time compared with other algorithms. First, in terms of

Table 7
Parameters of the comparison algorithms.

Name	Parameter	Value
RNSA	The initial temperature	1000
	Cooling coefficient	0.95
	Markov chain length	50
	Iterations	100/200/500
GVNS	Neighborhood structure	Same as TGVNSA
	Greedy search depth	5
	Iterations	100/200/500
IAGA	Population size	100
	Max/min Crossover Probability	0.9/0.4
	Crossover operator	0.6
	Max/min mutation probability	0.1/0.005
	Mutation Operator	0.025
	Iterations	100/200/500

number of pod retrieval times, the RNSA and GVNS algorithms have significantly lower accuracy compared to TGVNSA. IAGA and TGVNSA perform similarly well, with only a 0.15 % average difference in their results. In comparing CQ values, RNSA and GVNS perform worse than TGVNSA. IAGA outperforms TGVNSA in small to medium systems but falls short in large-scale systems. It's important to note that the number of pod retrieval times and CQ values are merely process variables and not the ultimate optimization objectives.

Upon comparing the comprehensive picking costs for each case, it is evident that RNSA has the lowest accuracy, exhibiting an average difference of 20.73 % from TGVNSA. GVNS shows a better performance than RNSA, with its average accuracy difference from TGVNSA being 6.9 %. In small and medium-sized systems, IAGA achieves nearly the same level of accuracy as TGVNSA. In scenarios of large systems, the IAGA algorithm exhibits a substantial decline in accuracy. In the 30×30 warehouse, IAGA performs 11.62 % worse than TGVNSA, as shown by the blue segment in the CPC column of Fig. 13. This is because, as the system size increases, the solution space expands significantly, making it difficult for 100 individuals in the population to cover most of the space, resulting in a decline in search quality. Increasing the population would greatly increase the computational load and time. Therefore, it is evident that the TGVNSA proposed in this paper demonstrates greater advantages in larger-scale scenarios.

Regarding CPU time, IAGA, which utilizes the concept of population optimization, requires considerably more time compared to other algorithms. Due to its use of the random neighborhood, RNSA outperforms both GVNS and TGVNSA in terms of CPU time efficiency.

Considering all comparison items, TGVNSA exhibits excellent performance in both computational accuracy and CPU time, especially in larger-scale scenarios where its advantages are even more pronounced.

5.4. Comparison of different strategies

The strategy adopted by the storage location assignment and order batching joint optimization model (SLAOB) proposed in this article combines the lowest pod recall frequency and the highest order relevance into a cost-optimal strategy. This section provides a comparison between SLAOB and various other strategies. For item location assignment, the strategies considered are Random Storage (RS), Classified Storage (CS), Optimal Clustering (OC), and Lowest Retrieval Times (LR). For pod location assignment, the strategies examined are Random Storage (RS), Classified Storage (CS), Decentralized Storage (DS), and Optimal Distance (OD). Additionally, the order batching strategies analyzed include Fixed Quantity (FQ), Time Window (TW), and Optimal Relevance (OR).

Different strategy combinations are compared with SLAOB, including CS-CS-FQ, CS-CS-OR, OC-CS-OR, LR-CS-OR, OC-OD-OR, and LR-OD-OR. The meaning of CS-CS-FQ is that the item location assignment phase adopts the Classified Storage (CS) strategy, the pod location assignment uses the Classified Storage (CS) strategy, and order batching adopts the Fixed Quantity (FQ) strategy, with the others following in a similar manner. Random Storage (RS) does not meet the requirements of this paper, Decentralized Storage (DS) aims to reduce congestion, and the Time Window method (TW) involves issues of order sequencing, therefore they are not discussed in this paper. When calculating the outcomes of the mentioned strategy combinations, if optimization is required, the TGVNSA algorithm framework introduced in this paper should be used. This ensures that the conditions for comparison are consistent.

Comparative experiments are carried out in the following two cases: Case 1: In a 10×10 system with configuration III, 40 orders are completed, each order contains 40 item requirements, and the types of items are 40; Case 2: In a 20×20 system with configuration I, 40 orders are completed, each order contains 40 item requirements, and the types of items are 40.

The comparison results of different strategy combinations are shown

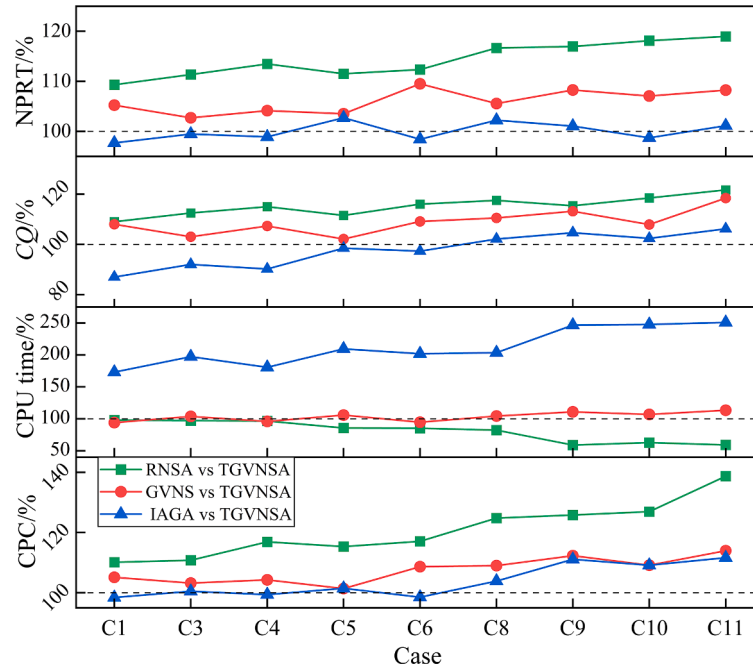


Fig. 13. Comparison of different algorithms.

in Fig. 14. Since the result of the RS strategy is much larger than other types, it is not marked in Fig. 14. Obviously, the OC and LR strategies are better than the RS and CS strategies in the item location assignment stage. As shown in Fig. 14, the OC-OD-OR and LR-OD-OR strategies are compared. For high storage densities like Case 1, the LR strategy proves more effective due to requiring fewer pods. Conversely, in more spacious environments such as Case 2, the wide choice of pod locations makes the OC strategy more efficient.

In the pod location assignment stage, OD is better than CS and RS. Comparing the results of the OC-OD-OR strategy combination in Case 1 and Case 2, it can also be concluded that the OD strategy has more significant advantages in a larger storage area.

During the order batching phase, the effect of the OR strategy is superior to FQ. However, the effect of the OR strategy also depends on the composition of the orders. The greater the similarity of the orders, the better the effect of using the OR strategy.

The SLAOB proposed in this paper is a joint optimization of the LR-OD-OR strategy combination. As can be seen in Fig. 14, the effect of SLAOB is better than other strategy combinations.

As can be seen in Fig. 14, the effect of SLAOB is significantly better than other strategy combinations and is also better than the LR-OD-OR strategy combination. The reason is that SLAOB is a joint optimization of the LR-OD-OR strategy combination, and each part of the LR-OD-OR strategy combination is optimized separately. The comprehensive picking costs of SLAOB in Case 1 decreased by 10.4 % compared to LR-

OD-OR and decreased by 15.9 % compared to OC-OD-OR. In Case 2, it decreased by 10.2 % compared to LR-OD-OR and decreased by 8.1 % compared to OC-OD-OR. It can be seen that under the storage configuration with higher depth, SLAOB has more outstanding optimization effects.

Based on the comparison results of various strategies in Fig. 14, the conclusions are summarized as follows: (i) During the item location assignment phase, the OC and LR strategies can bring the most significant optimization effect improvements, and the comprehensive picking costs are on average 22.71 % and 28.94 % lower than the CS strategy. (ii) During the pod location assignment phase, the OD strategy can bring the most significant optimization effect improvements, and the comprehensive picking costs are on average 31.22 % to 47.08 % lower than the CS strategy. (iii) Joint optimization has better effects than individual optimization at each stage, it can reduce the comprehensive picking costs by 11.46 %.

6. Conclusions

This paper studies the joint optimization of storage location assignment and order batching in RMFS, taking into account dynamic storage depth and surplus items. Firstly, a joint optimization model of storage location assignment and order batching is established. The model is divided into two stages. The first stage is the item location assignment optimization model, which is used to describe the types and quantities of

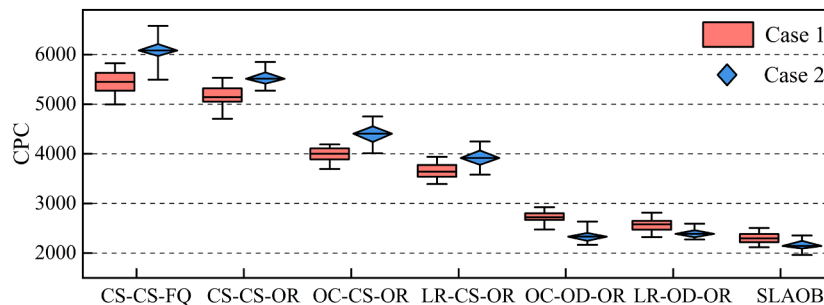


Fig. 14. Comparison of different strategy combinations.

items placed in each slot on each pod. The second stage is the joint optimization model of pod location assignment and order batching, which is used to describe the coordinates of each pod, the number of order batches, and the order combinations contained in each batch. Given the unique nature of the problem, a two-stage greedy variable neighborhood simulated annealing algorithm (TGVNSA) is proposed to solve the model. Finally, numerical experiments prove that the algorithm can effectively solve the established model. TGVNSA is compared with other algorithms in terms of number of pod retrieval times, comprehensive picking costs, CPU time, and CQ value. The results show that the overall performance of TGVNSA is superior to that of other algorithms. By comparing the strategy used in this paper with other strategies, it has been demonstrated that, when considering dynamic storage depth and surplus items, the optimization effect of the proposed method in this paper is significantly better than other strategy combinations. Moreover, under the same combination of strategies, the joint optimization method proposed in this paper reduces the comprehensive picking costs by 11.46 % compared to the separate optimization approach.

This paper successfully achieves the joint optimization of storage location assignment and order batching in RMFS. Experimental results demonstrate the effectiveness of the model and algorithm. However, for computational convenience, Manhattan distance is used to describe the proximity between pods and the picking station, which simplifies the actual travel paths of AGVs. Future research on task allocation and path planning in RMFS could also consider dynamic depth and remaining item quantities. For the issue of pod retrieval in multi-depth areas, collaborative handling by multiple robots could be explored to enhance system efficiency. Additionally, an attempt could be made to achieve full-process joint optimization for RMFS by simultaneously considering storage location assignment, order batching, task allocation, and path planning, and designing algorithms that incorporate AI decision-making to increase solution reliability and efficiency.

CRedit authorship contribution statement

Zhi Liu: Writing – original draft, Validation, Methodology, Investigation. **Jiansha Lu:** Writing – review & editing, Supervision, Resources, Methodology. **Chenhao Ren:** Writing – review & editing, Validation. **Jun Chen:** Validation. **Zhilong Xu:** Visualization. **Guoli Zhao:** Writing – review & editing, Resources.

Acknowledgements

This article is supported by Zhejiang Science and Technology Plan Project (Grant No. 2018C01003, 2024C01208).

Data availability

Data will be made available on request.

References

- Azadeh, K., De Koster, R., & Roy, D. (2019). Robotized and automated warehouse systems: Review and recent developments. *Transportation Science*, 53(4), 917–945. <https://doi.org/10.1287/trsc.2018.0873>
- Barros, I. R. D., & Nascimento, T. P. (2021). Robotic Mobile Fulfillment systems: A survey on recent developments and research opportunities. *Robotics and Autonomous Systems*, 137, 15. <https://doi.org/10.1016/j.robot.2021.103729>
- Boysen, N., Briskorn, D., & Emde, S. (2021). Parts-to-picker based order processing in a rack-moving mobile robots environment. *European Journal of Operational Research*, 262(2), 550–562. <https://doi.org/10.1016/j.ejor.2017.03.053>
- Boysen, N., de Koster, R., & Weidinger, F. (2019). Warehousing in the e-commerce era: A survey. *European Journal of Operational Research*, 277(2), 396–411. <https://doi.org/10.1016/j.ejor.2018.08.023>
- Cezik, T., Graves, S. C., & Liu, A. C. (2022). Velocity-based stowage policy for a semiautomated fulfillment system. *Production and Operations Management*, 16. <https://doi.org/10.1111/poms.13745>
- Cheng, B. Y., Wang, L. J., Tan, Q., & Zhou, M. (2024). A deep reinforcement learning hyper-heuristic to solve order batching problem with mobile robots. *Applied Intelligence*, 54(9–10), 6865–6887. <https://doi.org/10.1007/s10489-024-05532-9>
- D'Andrea, R., Wurman, P., & IEEE. (2008, Nov 10–11). *Future challenges of coordinating hundreds of autonomous vehicles in distribution facilities*. Paper presented at the IEEE International Conference on Technologies for Practical Robot Applications, Woburn, MA.
- Ding, T. R., Zhang, Y. K., Wang, Z., & Hu, X. P. (2024). Velocity-based rack storage location assignment for the unidirectional Robotic Mobile Fulfillment system. *Transportation Research Part E-Logistics and Transportation Review*, 186, 23. <https://doi.org/10.1016/j.tre.2024.103533>
- Gabellini, M., Calabrese, F., Regattieri, A., Loske, D., & Klumpp, M. (2024). A hybrid approach integrating genetic algorithm and machine learning to solve the order picking batch assignment problem considering learning and fatigue of pickers. *Computers & Industrial Engineering*, 191, 15. <https://doi.org/10.1016/j.cie.2024.110175>
- Gil-Borrás, S., Pardo, E. G., Alonso-Ayuso, A., & Duarte, A. (2021). A heuristic approach for the online order batching problem with multiple pickers. *Computers & Industrial Engineering*, 160, 20. <https://doi.org/10.1016/j.cie.2021.107517>
- Gil-Borrás, S., Pardo, E. G., Jiménez, E., & Sörensen, K. (2024). The time-window strategy in the online order batching problem. *International Journal of Production Research*, 62(12), 4446–4469. <https://doi.org/10.1080/00207543.2023.2263884>
- Hansen, P., & Mladenovic, N. (2001). Variable neighborhood search: Principles and applications. *European Journal of Operational Research*, 130(3), 449–467. [https://doi.org/10.1016/s0377-2217\(00\)00100-4](https://doi.org/10.1016/s0377-2217(00)00100-4)
- Hausman, W. H., Schwarz, L. B., & Graves, S. C. (1976). Optimal storage assignment in automatic warehousing systems. *Management Science*, 22(6), 629–638. <https://doi.org/10.1287/mnsc.22.6.629>
- Henn, S. (2012). Algorithms for on-line order batching in an order picking warehouse. *Computers & Operations Research*, 39(11), 2549–2563. <https://doi.org/10.1016/j.cor.2011.12.019>
- Keung, K. L., Lee, C., & Ji, P. (2022). Industrial internet of things-driven storage location assignment and order picking in a resource synchronization and sharing-based Robotic Mobile Fulfillment system. *Advanced Engineering Informatics*, 52, 19. <https://doi.org/10.1016/j.aei.2022.101540>
- Keung, K. L., Lee, C. K. M., & Ji, P. (2021). Data-driven order correlation pattern and storage location assignment in robotic mobile fulfillment and process automation system. *Advanced Engineering Informatics*, 50, 15. <https://doi.org/10.1016/j.aei.2021.101369>
- Kim, H. J., Pais, C., & Shen, Z. J. M. (2020). Item assignment problem in a Robotic Mobile Fulfillment system. *IEEE Transactions on Automation Science and Engineering*, 17(4), 1854–1867. <https://doi.org/10.1109/tase.2020.2979897>
- Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science (New York, N.Y.)*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>
- Kong, X. T. R., Zhu, M. H., Liu, Y., Qin, K. D., & Huang, G. Q. (2023). An advanced order batching approach for automated sequential auctions with forecasting and postponement. *International Journal of Production Research*, 61(12), 4180–4195. <https://doi.org/10.1080/00207543.2021.2022234>
- Lamballais, T., Roy, D., & De Koster, M. B. M. (2017). Estimating performance in a Robotic Mobile Fulfillment system. *European Journal of Operational Research*, 256(3), 976–990. <https://doi.org/10.1016/j.ejor.2016.06.063>
- Lanza, G., Passacantando, M., & Scutellà, M. G. (2022). Assigning and sequencing storage locations under a two level storage policy: Optimization model and matheuristic approaches x2729. *Omega-International Journal of Management Science*, 108, 17. <https://doi.org/10.1016/j.omega.2021.102565>
- Lee, I. G., Chung, S. H., & Yoon, S. W. (2020). Two-stage storage assignment to minimize travel time and congestion for warehouse order picking operations. *Computers & Industrial Engineering*, 139, 13. <https://doi.org/10.1016/j.cie.2019.106129>
- Li, H. R., Liu, J. S., Hu, P., & Zhou, H. (2024). Solving the storage location assignment of large-scale automated warehouse based on dynamic vortex search algorithm. *Swarm and Evolutionary Computation*, 91, 27. <https://doi.org/10.1016/j.swevo.2024.101725>
- Li, X. W., Hua, G. W., Huang, A. Q., Sheu, J. B., Cheng, T. C. E., & Huang, F. Q. (2020). Storage assignment policy with awareness of energy consumption in the Kiva mobile fulfillment system. *Transportation Research Part E-Logistics and Transportation Review*, 144, 18. <https://doi.org/10.1016/j.tre.2020.102158>
- Li, Y., Wan, Y., Zhang, Y., & Kuang, H. (2022). Path planning for warehouse robot based on the artificial bee colony-adaptive genetic algorithm. *Chinese Journal of Scientific Instrument*, 43(4), 282–290.
- Lin, C. C., Kang, J. R., Hou, C. C., & Cheng, C. Y. (2016). Joint order batching and picker Manhattan routing problem. *Computers & Industrial Engineering*, 95, 164–174. <https://doi.org/10.1016/j.cie.2016.03.009>
- Liu, M. Y., & Poh, K. L. (2023). E-commerce warehousing: An efficient scattered storage assignment algorithm with bulky locations. *Computers & Industrial Engineering*, 181, 18. <https://doi.org/10.1016/j.cie.2023.109236>
- Ma, Z. Q., Wu, G. H., Ji, B., Wang, L., Luo, Q. Z., & Chen, X. J. (2023). A Novel scattered storage policy considering commodity classification and correlation in Robotic Mobile Fulfillment systems. *IEEE Transactions on Automation Science and Engineering*, 20(2), 1020–1033. <https://doi.org/10.1109/tase.2022.3178934>
- Menéndez, S., Pardo, E. G., Alonso-Ayuso, A., Molina, E., & Duarte, A. (2017). Variable neighborhood search strategies for the order batching problem. *Computers & Operations Research*, 78, 500–512. <https://doi.org/10.1016/j.cor.2016.01.020>
- Mirzaei, M., Zaepour, N., & de Koster, R. B. M. (2022). How to benefit from order data: Correlated dispersed storage assignment in robotic warehouses. *International Journal*

- of *Production Research*, 60(2), 549–568. <https://doi.org/10.1080/00207543.2021.1971787>
- Momeni, M. A., & Jain, V. (2024). An integrated model for the production planning and the allocation of locations to items in a warehouse. *Annals of Operations Research*, 35. <https://doi.org/10.1007/s10479-024-06085-3>
- Nigam, S., Roy, D., de Koster, R., & Adan, I. J. (2014). Analysis of class-based storage strategies for the mobile shelf-based order pick system.
- Onal, S., Zhang, J. R., & Das, S. (2017). Modelling and performance evaluation of explosive storage policies in internet fulfilment warehouses. *International Journal of Production Research*, 55(20), 5902–5915. <https://doi.org/10.1080/00207543.2017.1304663>
- Roy, D., Nigam, S., de Koster, R., Adan, I., & Resing, J. (2019). Robot-storage zone assignment strategies in mobile fulfillment systems. *Transportation Research Part E-Logistics and Transportation Review*, 122, 119–142. <https://doi.org/10.1016/j.tre.2018.11.005>
- Scholz, A., Schubert, D., & Wäscher, G. (2017). Order picking with multiple pickers and due dates - Simultaneous solution of order batching, batch assignment and sequencing, and picker routing problems. *European Journal of Operational Research*, 263(2), 461–478. <https://doi.org/10.1016/j.ejor.2017.04.038>
- Tabrizi, A. M., Vahdani, B., Etebari, F., & Amiri, M. (2023). A three-stage model for clustering, storage, and joint online order batching and picker routing problems: Heuristic algorithms. *Computers & Industrial Engineering*, 179, 27. <https://doi.org/10.1016/j.cie.2023.109180>
- Wang, Y. Y., Man, R. J., Zhao, W. M., Zhang, H. L., & Zhao, H. (2022). Storage assignment optimization for fishbone Robotic Mobile Fulfillment systems. *Complex & Intelligent Systems*, 8(6), 4587–4602. <https://doi.org/10.1007/s40747-021-00597-2>
- Weidinger, F., & Boysen, N. (2018). Scattered storage: How to distribute stock keeping units all around a mixed-shelves warehouse. *Transportation Science*, 52(6), 1412–1427. <https://doi.org/10.1287/trsc.2017.0779>
- Weidinger, F., Boysen, N., & Briskorn, D. (2018). Storage assignment with rack-moving M bile robots in KIVA warehouses. *Transportation Science*, 52(6), 1479–1495. <https://doi.org/10.1287/trsc.2018.0826>
- Xiang, X., Liu, C. C., & Miao, L. X. (2018). Storage assignment and order batching problem in Kiva mobile fulfillment system. *Engineering Optimization*, 50(11), 1941–1962. <https://doi.org/10.1080/0305215x.2017.1419346>
- Yang, P., Jin, G., & Duan, G. F. (2022). Modelling and analysis for multi-deep compact robotic mobile fulfillment system. *International Journal of Production Research*, 60(15), 4727–4742. <https://doi.org/10.1080/00207543.2021.1936264>
- Yang, P., Miao, L. X., Xue, Z. J., & Qin, L. (2015). An integrated optimization of location assignment and storage/retrieval scheduling in multi-shuttle automated storage/retrieval systems. *Journal of Intelligent Manufacturing*, 26(6), 1145–1159. <https://doi.org/10.1007/s10845-013-0846-7>
- Yang, P., Miao, L. X., Xue, Z. J., & Ye, B. (2015). Variable neighborhood search heuristic for storage location assignment and storage/retrieval scheduling under shared storage in multi-shuttle automated storage/retrieval systems. *Transportation Research Part E-Logistics and Transportation Review*, 79, 164–177. <https://doi.org/10.1016/j.tre.2015.04.009>
- Yang, P., Peng, Y. F., Ye, B., & Miao, L. X. (2017). Integrated optimization of location assignment and sequencing in multi-shuttle automated storage and retrieval systems under modified 2<i>n</i>-command cycle pattern. *Engineering Optimization*, 49(9), 1604–1620. <https://doi.org/10.1080/0305215x.2016.1261128>
- Yang, P., Zhao, Z. J., & Guo, H. J. (2020). Order batch picking optimization under different storage scenarios for e-commerce warehouses. *Transportation Research Part E-Logistics and Transportation Review*, 136, 18. <https://doi.org/10.1016/j.tre.2020.101897>
- Yang, P., Zhao, Z. J., & Shen, Z. J. M. (2021). A flow picking system for order fulfillment in e-commerce warehouses. *IIE Transactions*, 53(5), 541–551. <https://doi.org/10.1080/24725854.2020.1772525>
- Yuan, R., Graves, S. C., & Cezik, T. (2019). Velocity-based storage assignment in semi-automated storage systems. *Production and Operations Management*, 28(2), 354–373. <https://doi.org/10.1111/poms.12925>
- Zarinchang, A., Lee, K., Avazpour, I., Yang, J., Zhang, D., & Knopf, G. K. (2024). Adaptive warehouse storage location assignment with considerations to order-picking efficiency and worker safety. *Journal of Industrial and Production Engineering*, 41(1), 40–59. <https://doi.org/10.1080/21681015.2023.2263009>
- Zhang, J., Wang, X. P., Chan, F. T. S., & Ruan, J. H. (2017). On-line order batching and sequencing problem with multiple pickers: A hybrid rule-based algorithm. *Applied Mathematical Modelling*, 45, 271–284. <https://doi.org/10.1016/j.apm.2016.12.012>
- Zhang, J., Zhang, N., Tian, L. K., Zhou, Z. J., & Wang, P. (2023). Robots? picking efficiency and pickers? energy expenditure: The item storage assignment policy in Robotic Mobile Fulfillment system. *Computers & Industrial Engineering*, 176, 16. <https://doi.org/10.1016/j.cie.2022.108918>
- Zhang, K., & Gao, C. H. (2023). Improved formulations of the joint order batching and picker routing problem. *International Journal of Production Research*, 61(21), 7386–7409. <https://doi.org/10.1080/00207543.2022.2149872>